

RESEARCH ARTICLE - METHODOLOGY

UFR-OSFA: Unified Feature Representation and Oppositional Structure Feature Alignment for Mixed-Project Heterogeneous Defect Prediction

Yifan Zou  | Huiqiang Wang  | Hongwu Lv  | Shuai Zhao 

College of Computer Science and Technology, Harbin Engineering University, Harbin, China

Correspondence: Yifan Zou (ivan@hrbeu.edu.cn)**Received:** 17 October 2024 | **Revised:** 29 May 2025 | **Accepted:** 13 August 2025**Funding:** The research was supported by the National Natural Science Foundation of China (No. 62272126), the Fundamental Research Funds for the Central Universities, Special Project for Industrial Foundation Reconstruction and High-Quality Development of Manufacturing Industry by the Ministry of Industry and Information Technology (No. TC220A04X-1), and the Natural Science Foundation of Heilongjiang Province (LH2021F001).**Keywords:** deep domain adaptation | deep learning | heterogeneous defect prediction | mixed project

ABSTRACT

Heterogeneous defect prediction (HDP) plays a crucial role in software engineering by enabling the early detection of software defects across projects with heterogeneous feature spaces. Recently, some mixed-project HDP (MP-HDP) methods have been proposed, which have demonstrated modest improvements in HDP performance. Nevertheless, existing MP-HDP approaches fail to address feature redundancy and distribution inconsistency simultaneously. To overcome these limitations, this paper proposes a novel MP-HDP approach, UFR-OSFA, based on unified feature representation and oppositional structural feature alignment. Concretely, UFR-OSFA first unifies these features by reducing the distribution differences between source and target projects through matching common features and the Hungarian algorithm based on the Kolmogorov–Smirnov (KS) test. Subsequently, utilizing a generator and two classifiers with oppositional structures, UFR-OSFA separates the features of the source project and clusters those of the target project, addressing the issue of conditional distribution mismatch and enhancing the model's generalization ability in the target project. Extensive experiments on 23 projects from five datasets demonstrate that the proposed approach performs better or comparably to baseline methods.

1 | Introduction

Software systems have grown significantly in scale and complexity in recent years, making software defects inevitable [1, 2]. As a critical technology in software engineering, software defect prediction (SDP) serves as an essential approach to enhancing software quality and reducing development costs [1, 3, 4]. Specifically, SDP helps development teams identify potential defect-prone modules early, thereby optimizing the allocation of testing resources, improving software quality, and lowering the cost of defect remediation. Research indicates that the cost of fixing software defects increases exponentially as the

development process progresses; early defect prediction can significantly reduce errors and system crashes caused by defects in later stages [5]. Consequently, SDP sweetens the reliability of software products, leading to greater user satisfaction and long-term sustainability of the software [6, 7].

However, traditional SDP methods focus mainly on within-project defect prediction (WPDP), which leverages historical defect data within a project to predict future defects in the same project [8–10]. Notably, the effectiveness of WPDP relies on sufficient historical data, which is often unavailable for new projects or early stages of development, significantly limiting its

practical application [11, 12]. Fortunately, cross-project defect prediction (CPDP) has been proposed, which constructs predictive models leveraging historical defect data from one or more source projects and applying them to the target project [12]. CPDP enables effective defect prediction even when the target project lacks historical data. Nevertheless, CPDP assumes that source and target projects share the same metrics or feature sets, meaning they have similar structures in the feature space. This assumption often proves unrealistic, as different software projects typically exhibit distinct metrics, development processes, coding styles, and defect distributions [13]. Accordingly, traditional CPDP models frequently underperform when applied to heterogeneous projects [14, 15].

To address this challenge, heterogeneous defect prediction (HDP) has emerged, aiming to address the feature heterogeneity between source and target projects [13–16]. HDP allows source and target projects to utilize heterogeneous features and transfer knowledge from the source project to the target project using techniques such as transfer learning. Recent years have seen significant advancements in HDP methods, with numerous studies proposing various models to mitigate the effects of project distribution differences. For example, Tong et al. [14] proposed KSETE, which remarkably improves heterogeneous project defect prediction performance by integrating class imbalance learning, multiple kernel spectral embedding, and transfer ensemble learning. Ren et al. [17] cleverly combine instance and feature adaptation techniques to effectively reduce distribution divergence between different projects, thus improving the accuracy of defect prediction. Bal et al. [18] improve defect prediction accuracy using chi-square tests and the Hungarian algorithm to select source data sets relevant to the target data set. Nonetheless, these methods typically rely solely on source project data and unlabeled target data, omitting the label information from the target data [19, 20]. To overcome this limitation, mixed-project heterogeneous defect prediction (MP-HDP) has been proposed [16, 21, 22], which constructs HDP models using source project data while effectively leveraging a small amount of labeled target data to enhance prediction performance. This approach proves especially beneficial when data availability is limited, especially during the initial phases of software testing [16]. This paper focuses on investigating MP-HDP and addressing its associated challenges.

1.1 | Motivation

Although MP-HDP has made some strides through feature transformation and domain adaptation techniques, achieving consistently high predictive performance across diverse and heterogeneous software projects remains a substantial challenge. In particular, issues such as redundant features in mismatched metric sets and inconsistent feature distributions between source and target domains continue to hinder effective knowledge transfer and generalization in HDP tasks.

1.1.1 | Redundant Feature in Heterogeneous Projects

In previous MP-HDP studies, most approaches have relied on feature transformation methods, which aim to map features

from both source and target projects in a shared feature space to reduce distribution discrepancies. However, the presence of redundant or irrelevant features in the original data can degrade the performance of the model [14, 15, 19, 23]. According to studies [24, 25], as a practical example, a NASA project contains 37 static code metrics (including Halstead Complexity Index, McCabe Complexity, etc.), while an Eclipse project uses 61 object-oriented metrics and change metrics. They have only one common line of code (LOC) for metrics. If one naively combines all the features for transformation, the model will be faced with a high-dimensional feature space filled with redundant or irrelevant metrics. Because Halstead metrics in NASA data have no meaning in Eclipse data, they will actually become noise when included in training [26]. Such noisy features increase the risk of overfitting the model and ultimately reduce the accuracy of defect prediction [25]. An alternative strategy known as feature selection matching has been proposed to address this problem. This approach aligns the feature sets by selecting and matching features from the source and target projects based on their mutual relevance, effectively filtering out irrelevant information on both sides [27–29]. Recognizing the complementary strengths of both methods, Chen et al. [13] advocate for an integrated approach that combines feature selection and transformation. Despite these promising insights, current MP-HDP research has yet to adopt such hybrid designs widely. Therefore, principled feature selection remains a critical first step in the construction of effective MP-HDP models.

1.1.2 | Inconsistent Feature Distributions

Beyond redundancy, a fundamental challenge in MP-HDP is the inconsistent distribution of defect features between source and target projects. This inconsistency arises from diverse factors, including differences in development environments (e.g., integrated development tools, version control systems), coding practices (e.g., adherence to style guides, code complexity), programming languages (e.g., Java vs. Python), project domains (e.g., embedded systems vs. web applications), and software metrics (e.g., lines of code vs. cyclomatic complexity). Such heterogeneity introduces distributional gaps that seriously impair model transferability and generalization across projects. This fundamental discrepancy gives rise to two specific manifestations that hamper MP-HDP:

1. **Conditional distribution mismatch:** This subchallenge arises when the conditional probability of a defect label given features, $P(Y|X)$, differs between the source and the target projects [30]. The conditional relationship can still diverge even if the marginal feature distributions are partially aligned through feature-transformation techniques. This mismatch confuses cross-project classifiers and often produces high false negative or false positive rates on target data and leads to a significant deterioration in predictive performance [21, 31]. The phenomenon is both theoretically predicted and empirically validated. For a practical example [32, 33], in the Safe project, defective modules tend to be larger (higher LOC) than clean ones, whereas in Velocity-1.4 the trend reverses-defective modules are typically smaller (lower LOC). Thus, a naive model trained in Safe (where “more

LOC = defect”) would misclassify many modules in Velocity-1.4. More broadly, metrics such as complexity or size are correlated with defect proneness in some projects but not in others, reflecting different coding practices and architectures [34]. This confirms the need for an explicit conditional-distribution alignment mechanism during the construction of the MP-HDP model.

2. **Decision boundary bias:** In MP-HDP, the abundance of labeled data in the source project frequently leads the MP-HDP model to overfit the feature distribution of the source project, resulting in a decision boundary biased toward the source domain [35]. When there are significant differences in feature distribution between the source and target domains, this decision-boundary bias directly weakens the generalization ability of the target domain [36]. DSKMDA [16] provides a practical case: on the Apache, Safe, and ZXing target projects in the ReLink dataset, which differ greatly from other datasets, even with integrated feature alignment and class rebalancing, DSKMDA’s G-mean, MCC, Popt, and ACC on the above projects are still surpassed by the unsupervised baseline ManualDown/Up. The decision threshold of ManualDown/Up is entirely based on the LOC distribution of the target project itself, indicating that DSKMDA’s decision boundary is still “pulled” toward the source domain by a large number of source project samples, failing to adapt to the target domain feature distribution. Therefore, relying solely on the alignment of the distribution without explicit boundary constraints tailored to the target domain can lead to performance degradation in scenarios with significant differences between the source and the target [36]. Recent studies have shown that adversarial conditional entropy optimization addresses this bias by clustering target features while dispersing source features, effectively balancing the influence of both domains during training [35, 37]. In general, explicitly optimizing the decision boundary during training remains a key driver in improving the adaptability and robustness of MP-HDP. This study is a practical advance in this direction.

Together, these challenges inhibit the effectiveness of MP-HDP models. An effective MP-HDP approach should integrate statistically grounded feature selection to eliminate redundant features and employ advanced alignment techniques to resolve feature distribution inconsistencies.

1.2 | Contribution

We propose UFR-OSFA, a novel MP-HDP approach based on unified feature representation (UFR) and oppositional structure feature alignment (OSFA), to tackle the aforementioned challenges. In the UFR stage, we first match the common features between the source and target projects to develop a model based on feature matching. We then employ the Kolmogorov–Smirnov (KS) test to evaluate the distributional similarity between the features of the source and target projects. Thereafter, the Hungarian algorithm determines the optimal matching subset of unique features between different projects, thereby reducing the distribution gap between the source and target projects.

In the subsequent OSFA stage, we construct a model consisting of a generator and two classifiers with opposing structures: a source project scattering classifier and a target project clustering classifier. The source project scattering classifier maximizes the conditional entropy of the source project features, promoting the source features to be more widely dispersed in the feature space. Conversely, the target project clustering classifier minimizes the conditional entropy of the target project samples, encouraging tighter clustering of the unlabeled samples in the target project within the feature space, thereby forming a clear class structure. By simultaneously clustering target features and dispersing source features, OSFA mitigates conditional distribution mismatch, leverages limited labeled target samples, and prevents the decision boundary from overfitting the source project’s feature distribution. This oppositional structure enhances cross-domain feature alignment and significantly improves the model’s generalization capability in the target project.

In general, this paper makes the following contributions:

1. To unify the features of the source and target projects, UFR-OSFA employs the KS test to assess the distributional similarity of features between them and applies the Hungarian algorithm to match the optimal subset of feature matches between the projects, thereby effectively reducing distributional differences. This is the first attempt to apply feature-matching selection techniques in the MP-HDP domain.
2. UFR-OSFA design an oppositional structure consisting of a generator and two classifiers (a source project scattering classifier and a target project clustering classifier). This structure addresses conditional distribution mismatch and decision boundary bias by maximizing the conditional entropy of the source project while minimizing that of the target project, thereby dispersing source features and clustering target features.
3. We conduct extensive experiments on 23 projects across five datasets. The results demonstrate that the proposed approach significantly outperforms existing benchmark methods across multiple metrics, including AUC, F -measure, MCC, and Popt. These findings validate the effectiveness and superiority of UFR-OSFA in MP-HDP.

The organization of this paper is as follows: Section 1.1 discusses research related to HDP, including traditional HDP methods and MP-HDP methods. Section 2 explains the proposed approach and its crucial implementation details. Section 3 outlines the experimental setup, including the research questions, datasets used, baseline models, parameter configurations, and evaluation metrics. In Section 4, we describe the experimental design and present the results. Section 5 gives discussions on UFR-OSFA. Thereon, Section 6 addresses potential threats to validity and their implications. Finally, Section 7 summarizes the main findings of this paper.

2 | Related Work

This section discusses existing HDP methods, which can be broadly categorized into two groups based on whether they

utilize labeled data from the target project [16, 21]: traditional HDP methods and MP-HDP methods.

2.1 | Traditional HDP Methods

Explicitly, the traditional HDP methods are classified into feature matching and feature transformation [13]. Feature matching-based HDP methods align heterogeneous features between source and target projects by selecting relevant features to reduce distributional differences. For example, Nam et al. [25] first perform feature selection on data from the source project to remove redundant or irrelevant features and then match the metrics from the source project with those of the target project through a metric matching algorithm to ensure that they are similar in distribution or relevance. Yu et al. [38] perform feature selection of the source project and the target project to obtain their feature distribution curves. Then, using the designed feature matching algorithm, the heterogeneous features of the source project are converted to match the features of the target project according to the distance between the distribution curves, and the feature migration is finally realized.

Differently, feature transformation-based HDP methods project heterogeneous datasets into a shared feature space or map source features to target features to minimize distribution discrepancies. Li et al. [39] mapped the source and target data to the high-dimensional feature space through a two-stage integrated learning (TSEL), firstly by employing the multicore correlation alignment (EMKCA) and then by a domain adaptation technique to align the source and target data space and align the distribution of source and target data by domain adaptation technique. Then, multiple EMKCA predictors are generated and integrated using the data resampling (RES) technique. Tong et al. [14] first mitigate the category imbalance problem by balancing the category distributions of the source data. Next, the kernel spectral embedding technique transforms the source and target data into a shared feature space. This process aims to align the distributions of the two datasets as closely as feasible while preserving their key characteristics. Finally, integrated learning constructs classifiers in multiple subspaces to improve prediction accuracy.

2.2 | Mixed-Project HDP Methods

Alternatively, MP-HDP involves constructing a predictive HDP model by combining labeled source project samples with a limited number of labeled target samples. Incorporating a small set of labeled target samples during the HDP process significantly enhances subsequent prediction accuracy. Despite its promise, research on MP-HDP remains relatively scarce. Li et al. [19] were pioneers in incorporating heterogeneous project data into HDP. They introduced a unique MP-HDP approach called CLSUP, which integrates heterogeneous dataset adaptation and cost-sensitive learning. This method initially transforms the source project's data into a subspace that mirrors the target project's distribution while preserving the source data's structural integrity. By implementing cost-sensitive learning, different costs are assigned to different classification errors, improving predictive accuracy. Li et al. [20] also presented a

multisource selection technique (MSMDA) based on a manifold discriminant alignment. MSMDA identifies multiple source projects with distributions akin to the target project and aligns both target and source projects into a comparable subspace. The MDA model requires some labeled target data during its development, which is categorized as the MP-HDP method. Niu et al. [16] extended the MDA method by initially employing SMOTE to address class imbalances in the source project. They then used kernel-manifold discriminant alignment to map data from both source and target projects into a mutual nonlinear subspace, addressing issues related to linear separability. Gong et al. [22] addressed HDP using conditional domain adversarial adaptation (CDAA). The CDAA framework consists of a generator, a discriminator, and a classifier. The generator aligns the feature space of the source project with that of the target project, the discriminator distinguishes between the source and target data, and the classifier accurately learns to classify the data labels. CDAA leverages the generative adversarial network (GAN) framework, integrating label information from both the source and a limited number of target projects to maintain classification accuracy during feature transformation.

In contrast to these MP-HDP methods, this paper proposes a novel MP-HDP approach based on unified feature representation and oppositional structure feature alignment. Our method aims to address domain distribution disparities and mitigate decision boundary biases toward the source project.

3 | Methodology

In this section, we detail our proposed MP-HDP approach, UFR-OSFA. Figure 1 illustrates the framework of UFR-OSFA.

1. **Unified feature representation:** This stage starts by matching common features between source and target projects to establish initial alignment in the feature space. Thereafter, the KS test is employed to measure the similarity between feature distributions of the source and target projects. For the features unique to each project, the Hungarian algorithm is applied for optimal subset matching, thereby reducing the discrepancy in data distribution between the source and target projects for more uniform bases for subsequent feature transformation and model training.
2. **Oppositional structure feature alignment:** A shared feature generator extracts features from the source and target projects. These features are then processed by two classifiers: a source project scattering classifier that maximizes the conditional entropy of the source project to ensure feature dispersion and prevent decision boundary bias toward the source project, and a target project clustering classifier that minimizes the conditional entropy of the target project to promote feature clustering and reduce distribution discrepancies between projects. The feature generator continuously refines feature representations during training through these opposing optimization processes, ultimately achieving effective alignment between heterogeneous projects.

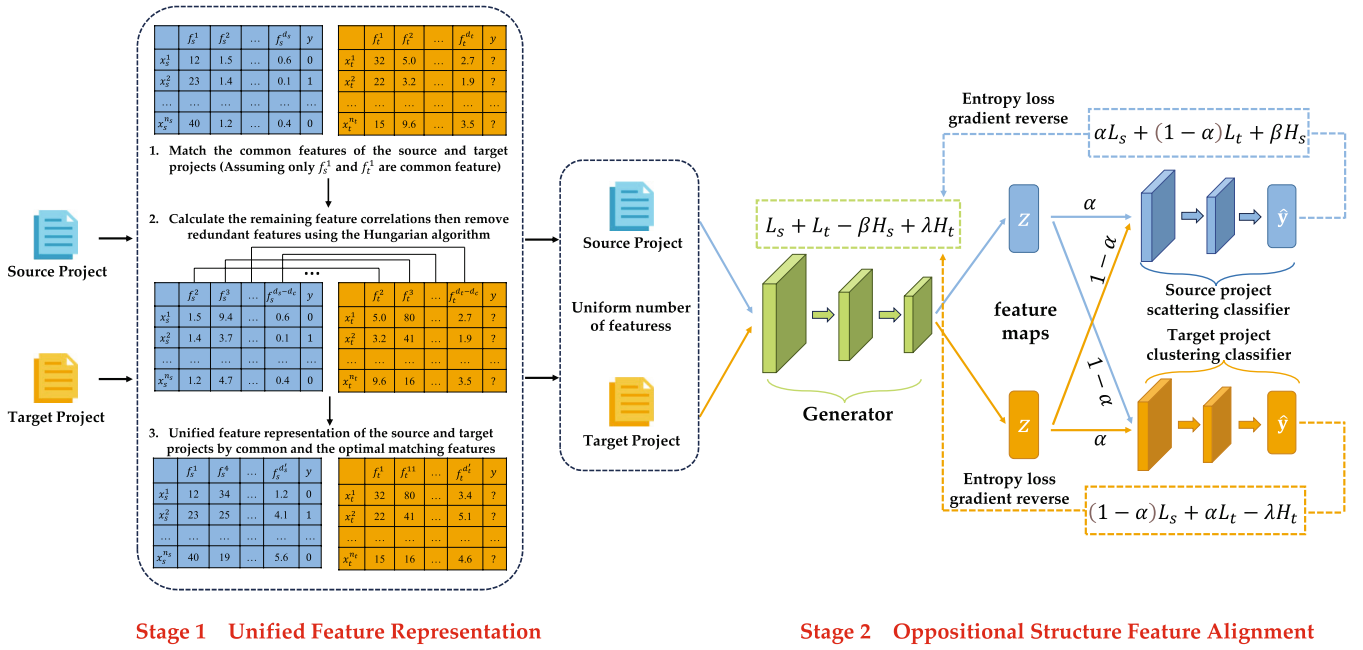


FIGURE 1 | The framework of UFR-OSFA.

The remainder of this section later defines key terminology used throughout the paper, followed by a detailed description of the UFR-OSFA method. For clarity, Table 1 summarizes the notations and their corresponding explanations.

3.1 | Notion

For ease of representation in our approach, we denote the source project as $D_S = \{X_S, y_S\}$, where $X_S = \{x_s^i\}_{i=1}^{n_s}$ and $y_S = \{y_s^i\}_{i=1}^{n_s}$. Here, n_s represents the number of samples in the source project. Each sample $x_s^i \in R^{d_s}$, where d_s denotes the number of features, and each sample is labeled as $y_s^i \in \{0, 1\}$. Similarly, following the definitions in prior MP-HDP papers [16, 17], the target project $D_T = \{X_T, y_T\}$ consists of a large unlabeled dataset X_U and a small labeled dataset X_L . Thus, $X_T = X_U \cup X_L$ and $y_T = y_U \cup y_L$. The unlabeled dataset X_U is defined as $X_U = \{x_u^i\}_{i=1}^{n_u}$, with corresponding labels $y_U = \{y_u^i\}_{i=1}^{n_u}$, where n_u is the number of unlabeled samples. The labeled dataset X_L is given by $X_L = \{x_l^i\}_{i=1}^{n_l}$, with labels $y_L = \{y_l^i\}_{i=1}^{n_l}$, where n_l is the number of labeled samples. The total number of samples in the target project is $n_t = n_u + n_l$, with $n_u \gg n_l$. Each sample in the target project, $x_u^i, x_l^i \in R^{d_t}$, is represented by d_t features, and their labels $y_u^i, y_l^i \in \{0, 1\}$. Note that in MP-HDP, the features of the source and target projects are heterogeneous, meaning $d_s \neq d_t$.

3.2 | Unified Feature Representation

Given the feature space of the source project and target project is defined as $F_S = \{f_s^1, f_s^2, \dots, f_s^{d_s}\}$ and $F_T = \{f_t^1, f_t^2, \dots, f_t^{d_t}\}$, respectively. We first identify the common features between the source and target projects, denoted as $F_c = \{f_{st}^1, f_{st}^2, \dots, f_{st}^{d_c}\}$, where d_c represents the number of common features, as

illustrated in Table 2. Thus, the set of unique features for the source and target projects is $F_{S_unique} = \{f_s^1, f_s^2, \dots, f_s^{d_s-d_c}\}$, $F_{T_unique} = \{f_t^1, f_t^2, \dots, f_t^{d_t-d_c}\}$, respectively. Our goal is to find the subset of features in F_{S_unique} and F_{T_unique} that have the maximum distributional similarity and no duplicate features. To achieve this, we employ the Hungarian algorithm [18] to find the optimal matching subset of unique features between different projects using dynamic thresholding.

In particular, we first establish a feature similarity score matrix, utilizing the KS test to calculate the similarity score, which is widely used in defect prediction [18, 40, 41]. For a source project and a target project, let p_{ij} denote the p -value of the KS-test between the i th feature in the source project's unique feature set and the j th feature in the target project's unique feature set. When two features exhibit different distributions, the p -value tends to be smaller, whereas similar features yield larger p -values. Due to the essence of the Hungarian algorithm lying in finding the minimum cost matching, the similarity score between the two features is scored as $Score_{ij} = 1 - p_{ij}$. This results in the following similarity matrix for the unique feature sets:

$$M = \begin{bmatrix} Score_{11} & Score_{12} & \dots & Score_{1(d_t-d_c)} \\ Score_{21} & Score_{22} & \dots & Score_{2(d_t-d_c)} \\ \vdots & \vdots & \ddots & \vdots \\ Score_{(d_s-d_c)1} & Score_{(d_s-d_c)2} & \dots & Score_{(d_s-d_c)(d_t-d_c)} \end{bmatrix} \quad (1)$$

where $Score_{ij}$ plays the role of a distance: smaller values indicate greater feature similarity. The Hungarian algorithm simplifies the matching computation by modifying the weight matrix to remove redundant feature pairs. The steps of the algorithm are as follows:

TABLE 1 | Notation definitions.

Symbol	Description
D_S	Source project dataset, $D_S = \{X_S, y_S\}$
X_S	Source project samples, $X_S = \{x_s^i\}_{i=1}^{n_s}$
y_S	Source project labels, $y_S = \{y_s^i\}_{i=1}^{n_s}$, where $y_s^i \in \{0, 1\}$
x_s^i	i th sample in the source project X_S , $x_s^i \in \mathbb{R}^{d_s}$
n_s	Number of source project samples
d_s	Number of source project features
D_T	Target project dataset, $D_T = \{X_T, y_T\}$
X_T	Target project samples, $X_T = X_U \cup X_L$
y_T	Target project labels, $y_T = y_U \cup y_L$, where $y_u^i, y_l^i \in \{0, 1\}$
X_U	Unlabeled target project samples, $X_U = \{x_u^i\}_{i=1}^{n_u}$
x_u^i	i th sample in the unlabeled target project X_U , $x_u^i \in \mathbb{R}^{d_t}$
n_u	Number of unlabeled target project samples
X_L	Labeled target project samples, $X_L = \{x_l^i\}_{i=1}^{n_l}$
x_l^i	i th sample in the labeled target project X_L , $x_l^i \in \mathbb{R}^{d_t}$
n_l	Number of labeled target project samples
n_t	Total number of target project samples, $n_t = n_l + n_u$
d_t	Number of target project features
F_S	Source project feature space, $F_S = \{f_s^1, \dots, f_s^{d_s}\}$
F_T	Target project feature space, $F_T = \{f_t^1, \dots, f_t^{d_t}\}$
F_c	Common features between source and target projects, $F_c = \{f_c^1, f_c^2, \dots, f_c^{d_c}\}$
d_c	Number of common features between source and target projects.
F_{S_unique}	Unique features in the source project, $F_{S_unique} = \{f_s^1, f_s^2, \dots, f_s^{d_s-d_c}\}$
F_{T_unique}	Unique features in the target project, $F_{T_unique} = \{f_t^1, f_t^2, \dots, f_t^{d_t-d_c}\}$
F_{S_match}	Subsets of optimally matched unique features in the source project, $F_{S_match} = \{f_s^1, f_s^2, \dots, f_s^r\}$
F_{T_match}	Subsets of optimally matched unique features in the target project, $F_{T_match} = \{f_t^1, f_t^2, \dots, f_t^r\}$
r	The number of final pairs matched
$F_{S_unified}$	Unified feature set of the source project, $F_{S_unified} = F_c \cup F_{S_match}$
$F_{T_unified}$	Unified feature set of the target project, $F_{T_unified} = F_c \cup F_{T_match}$

Step 1. Row subtraction: For each row, subtract the smallest element of the row, so that at least one element of each row is 0.

Step 2. Column subtraction: For each column, subtract the smallest element of the column so that at least one element of each column is 0.

Step 3. Preliminary allocation: After subtraction of rows and columns, try to find an independent set of zero elements so that each row and column has at most one zero covered (this is a preliminary allocation attempt).

Step 4. Optimality check: If n independent zeros can be found (i.e., a perfect match is found), the algorithm ends, and this is the optimal solution. If not enough independent zeros can be found (i.e., a perfect match is not found), the adjustment needs to continue.

Step 5. Matrix adjustment: Find the smallest element uncovered in the matrix, denoted min . Subtract min from all uncovered elements and add min to any element covered by both lines. This step creates new zeros, increasing the number of possible matches.

Step 6. Repeat steps 3–5 until a perfect match is found.

To facilitate understanding, let us take an example to understand the steps of the Hungarian algorithm. Suppose now that there is a distributional similarity score matrix M_1 , which is expressed as

$$M_1 = \begin{bmatrix} 0.9 & 0.2 & 0.7 \\ 0.6 & 0.4 & 0.3 \\ 0.5 & 0.8 & 0.1 \end{bmatrix} \quad (2)$$

where

$$M_1 = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

$$M_1 = [Y_1 \ Y_2 \ Y_3]$$

For clarity, M_1 is a single matrix and the alternative forms shown (e.g., labeling the rows of M_1 as X_1, X_2, X_3 and the columns of M_1 as Y_1, Y_2, Y_3 in the example) are equivalent representations to illustrate its structure.

Following this, we perform **Step 1**. In each row, and subtract the minimum value of that row, with the aim of having at least one element of 0 in each row, to obtain the matrix M_2 .

$$M_2 = \begin{bmatrix} 0.7 & 0 & 0.5 \\ 0.3 & 0.1 & 0 \\ 0.4 & 0.7 & 0 \end{bmatrix} \quad (3)$$

We then complete **Step 2**. By subtracting from each column the minimum value of that column, with the aim of having at least one element of 0 in each column, to obtain the matrix M_3 .

TABLE 2 | Number of common features between datasets.

Dataset	AEEM	ReLink	NASA	PROMISE	SOFTLAB
AEEM	61	1	1	8	1
ReLink	1	26	4	1	4
NASA	1	4	37	1	28
PROMISE	8	1	1	20	1
SOFTLAB	1	4	28	1	29

$$M_3 = \begin{bmatrix} 0.4 & 0 & 0.5 \\ 0 & 0.1 & 0 \\ 0.1 & 0.7 & 0 \end{bmatrix} \quad (4)$$

At this point, we have found three independent zeros (one and only one zero in each row and column), which means that we have successfully found a perfect matching pair, that is, (X_1, Y_2) , (X_2, Y_1) , (X_3, Y_3) . After using the above Hungarian algorithm for M , we can obtain a series of optimal matching feature pairs, and each feature pair has a corresponding similarity score. Because each source (target) feature can only appear once in the best matching feature pairs, the maximum number of matched pairs is limited by the number of unique features in the smaller of the two unique feature sets. We express this upper bound as $\min[d_s - d_c, d_t - d_c]$. Intuitively, it guarantees that it prevents many-to-one pairings and keeps the source and target projects end up with exactly the same dimensionality, which is a prerequisite for subsequent matrix operations in OSFA. This ensures that the number of matched features does not exceed the smaller set of unique features from either project.

Next, we analyze the distribution of these scores to determine the optimal threshold dynamically, and finally, the most similar feature pairs will be filtered through the threshold among the feature pairs. We use the mean value to denote the average of all scores, which reflects the overall similarity between the features in the dataset.

$$\mu = \frac{1}{n} \sum_{i=1}^n Score_i \quad (5)$$

where n is the number of optimal matching feature pairs.

Also, we use the standard deviation σ to indicate the dispersion of the similarity scores, reflecting the volatility of the similarity between different pairs of features.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (Score_i - \mu)^2} \quad (6)$$

A dynamic threshold is set on the basis of the mean and standard deviation of the similarity score. The dynamic threshold can be calculated using the following formula, where k is usually set to 1.

$$T = \mu - k \times \sigma \quad (7)$$

Because a lower $Score_{ij}$ indicates higher similarity (due to $Score_{ij} = 1 - p_{ij}$), we retain only pairs with similarity scores below T , discarding pairs with $Score_{ij} \geq T$. This thresholding ensures that only feature pairs with sufficiently high distributional similarity are selected. Consequently, the final number of pairs matched, denoted r , is less than or equal to $\min[d_s - d_c, d_t - d_c]$, depending on how many pairs satisfy the threshold condition. Therefore, the subsets of optimally matched unique features are defined as $F_{S_match} = \{f_s^1, f_s^2, \dots, f_s^r\}$ and $F_{T_match} = \{f_t^1, f_t^2, \dots, f_t^r\}$. The unified features subsets are then constructed by combining the common features and the unique matched features: $F_{S_unified} = F_c \cup F_{S_match}$ and $F_{T_unified} = F_c \cup F_{T_match}$.

3.3 | Oppositional Structure Feature Alignment

To address the issues of conditional distribution mismatch and decision boundary bias, OSFA employs a feature generator to extract features from both the source and the target projects and optimize these features using two opposite classifiers.

3.3.1 | Model Component

Among them, the feature generator G is a core module of OSFA, responsible for extracting feature representations from input source and target project data. For each input sample x , the feature generator outputs a feature vector $f \in R^d$, which is the basis for subsequent classification tasks and training. Specifically, the generator must produce more decentralized features for the source project to mitigate decision boundary bias, while generating tightly clustered features for the target project to improve classification accuracy. Formally, the generator's output is expressed as

$$f = G(x, \Theta_G) \quad (8)$$

where Θ_G is a parameter of the feature generator. By employing a shared network of generators, the feature distributions of the source and target projects can be aligned in a unified feature space, which ultimately allows the model to be effectively generalized over the target project by optimizing the decision boundary.

In addition, the source project scattering classifier $C_s(f, \Theta_{C_s})$ reduces the overdependence of the decision boundary on the source project samples by scattering the source project features and inducing the feature points of the source project samples to be dispersed. This scattering process is achieved by maximizing

the conditional entropy of the source project features, resulting in a more dispersed feature distribution in the source project, which is conducive to enhancing the model's generalization ability in cross-project tasks. The source project scattering classifier is used to classify the features of the source project generated by the generator G . The output of the classifier C_s is calculated by the Sigmoid function with the following formula:

$$p_1(y|x) = \sigma(C_s(f(x), \Theta_{C_s})) \quad (9)$$

where Θ_{C_s} is the parameter of the classifier C_s .

Correspondingly, the target project clustering classifier $C_t(f, \Theta_{C_t})$ aims to make the features of the target project cluster together as much as possible. By minimizing the conditional entropy of the target project samples, the sample features of the target project are prompted to cluster to enhance intra-class consistency and increase inter-class separation. Similarly to the source project scattering classifier, the target project clustering classifier is used to binary classify the features of the target project generated by the generator G . The output of the classifier C_t is also calculated by the Sigmoid function:

$$p_2(y|x) = \sigma(C_t(f(x), \Theta_{C_t})) \quad (10)$$

where Θ_{C_t} is a parameter of the classifier C_t .

3.3.2 | Loss Function Design

3.3.2.1 | Supervised Classification Loss. The supervised classification loss evaluates the accuracy of the model's classification on labeled data from both the source and the target projects. Its purpose is to enhance the classifier's ability to categorize samples in both the source and target projects correctly. This loss consists of two components: the source project supervised classification loss L_s and the target project supervised classification loss L_t .

Concretely, L_s quantifies the model's classification error on labeled samples from the source project, and because the source project samples come with complete labels, we can use these labels for supervised learning of the classifier's predictions. The model learns different classes of features in the source project by minimizing the loss of supervised classification in the source project. We use the cross-entropy loss for binary classification to calculate, for each source project sample, the source project supervised classification loss (x_s^i, y_s^i) , the supervised classification loss is calculated as

$$L_s = -\frac{1}{n_s} \sum_{i=1}^{n_s} [y_s^i \log(p_1(y=1|x_s^i)) + (1-y_s^i) \log(1-p_1(y=1|x_s^i))] \quad (11)$$

Similarly, L_t measures the model's classification error on labeled samples in the target project. Although there may be fewer labeled samples in the target project, the model can learn how to classify the small number of samples in the target project by minimizing the loss of supervised classification in the target

project. Similarly to the source project supervised classification loss, the target project supervised classification loss is computed through the cross-entropy loss. For each target project labeled sample (x_t^i, y_t^i) , the supervised classification loss is computed as

$$L_t = -\frac{1}{n_t} \sum_{i=1}^{n_t} [y_t^i \log(p_2(y=1|x_t^i)) + (1-y_t^i) \log(1-p_2(y=1|x_t^i))] \quad (12)$$

3.3.2.2 | Conditional Entropy Loss. Conditional entropy loss guides the scattering or clustering of features by measuring the uncertainty in the predictive distribution of the classifier output. For the source project, maximizing the conditional entropy promotes feature dispersion, whereas for the target project, minimizing it encourages feature clustering. By adjusting the magnitude of the entropy, the feature structure in the source and target projects can be controlled. The conditional entropy loss includes the target project conditional entropy loss H_t and the source project conditional entropy loss H_s .

To be specific, H_t facilitates the clustering of features of unlabeled samples in the target project, ensuring the feature vectors align closely with their respective class centers, thus improving the certainty of classification. For each target project unlabeled sample x_u^i , the conditional entropy loss is computed as:

$$H_t = -\frac{1}{n_u} \sum_{i=1}^{n_u} [p_2(y=1|x_u^i) \log(p_2(y=1|x_u^i)) + (1-p_2(y=1|x_u^i)) \log(1-p_2(y=1|x_u^i))] \quad (13)$$

In contrast to clustering in the target project, H_s serves to drive source project features to be more dispersed. By maximizing the conditional entropy of the source project samples, this approach reduces the decision boundary's overdependence on the source project samples. As a result, the features of the source project are distributed more uniformly across the feature space rather than being overly concentrated near the decision boundary. For each source project sample x_s^i , H_s is calculated as

$$H_s = -\frac{1}{n_s} \sum_{i=1}^{n_s} [p_1(y=1|x_s^i) \log(p_1(y=1|x_s^i)) + (1-p_1(y=1|x_s^i)) \log(1-p_1(y=1|x_s^i))] \quad (14)$$

3.3.2.3 | Total Loss. Overall, the total loss of the source project scattering classifier C_s combines a supervised classification loss in the source project, a supervised classification loss in the target project, and a conditional entropy loss in the source project. This loss function ensures that the classifier learns to classify correctly on the source project while increasing the dispersion of the features of the source project. The loss function is defined as

$$\Theta_{C_s}^* = \arg \min_{\Theta_{C_s}} (\alpha L_s + (1-\alpha) L_t + \beta H_s) \quad (15)$$

where α is a weight hyperparameter controlling the weight of supervised classification loss in the source and target projects. The weight hyperparameter β controls the weight of conditional entropy loss in the source project.

Meanwhile, the total loss of the target project clustering classifier C_t consists of a supervised classification loss in the source project, a supervised classification loss in the target project and a conditional entropy loss in the target project. This loss function is designed to motivate the classifier to correctly classify the target project while minimizing the uncertainty of the features in the target project, thus motivating feature clustering. The loss function is expressed as

$$\Theta_{C_t}^* = \arg \min_{\Theta_{C_t}} ((1 - \alpha)L_s + \alpha L_t - \lambda H_t) \quad (16)$$

where λ is the weight hyperparameter that controls the weight of conditional entropy loss in the target project.

Additionally, the generator G optimizes a weighted combination of supervised classification loss and conditional entropy loss for both the source and target projects. This guides the generator to produce feature representations that fulfill two opposing objectives: (1) enabling both classifiers to correctly classify the generated features by minimizing supervised classification losses and (2) generating features that satisfy the scattering requirement for the source project (by maximizing H_s) and the clustering requirement for the target project (by minimizing H_t). The total loss of the generator is given by

$$\Theta_G^* = \arg \min_{\Theta_G} (L_s + L_t - \beta H_s + \lambda H_t) \quad (17)$$

3.3.3 | Training Procedure

In the OSFA framework, the training of the generator and the two classifiers is alternated, and the oppositional training is realized through the gradient reversal mechanism. The specific steps are as follows:

1. Stochastic initialization: The parameters of generator G , source project scattering classifier C_s , and target project clustering classifier C_t are first initialized.
2. Optimize the classifiers: In each iteration round, first, fix the generator parameters G and optimize the parameters of the source project scattering classifier C_s and the target project clustering classifier C_t .

When optimizing C_s , the minimization formula Equation (15).

Minimize the formula Equation (16) when optimizing C_t .

3. Optimizing the generators: The parameters of the two classifiers C_s and C_t are fixed, and the parameters of the generator G are optimized by back-propagating the gradient so that it generates features that contribute to the opposing targets. The parameters of the generator are updated by minimizing Equation (17).

Through repeated iterations of the above steps, the OSFA framework gradually optimizes the generators and classifiers so that the source and target project features achieve scattering and clustering, respectively, and ultimately effectively align heterogeneous domain features.

3.4 | Time Complexity Analysis

To assess the practical applicability of UFR-OSFA, we analyze its computational complexity. In the UFR stage, the KS test computes correlations between the unique features of the source $d_s - d_c$ and the target $d_t - d_c$, each with n_s and n_t samples. The KS test leads to a total complexity of $O((d_s - d_c) \cdot (d_t - d_c) \cdot (n_s + n_t))$ for all pairs. The Hungarian algorithm then matches unique features with a complexity of $O(d_s - d_c, d_t - d_c)^3$. In the OSFA stage, a feature generator and two classifiers (source scattering and target clustering) are trained for T iterations. Assuming a neural network with L layers and H hidden units per layer, the training complexity is $O(T \cdot (n_s + n_t) \cdot H^2 \cdot L)$. The total complexity of the training for UFR-OSFA is $O((d_s - d_c) \cdot (d_t - d_c) \cdot (n_s + n_t)) + O(d_s - d_c, d_t - d_c)^3 + O(T \cdot (n_s + n_t) \cdot H^2 \cdot L)$.

4 | Experiment Setup

This section presents the research questions, the details of the dataset, the baseline methods, and the evaluation metrics.

4.1 | Research Questions

To evaluate UFR-OSFA, we answer the following three research questions:

1. RQ-1: Does the UFR-OSFA approach perform better than other baseline methods for HDP?

We assess the effectiveness of UFR-OSFA by comparing it with multiple baseline methods across 23 projects, using four performance metrics for evaluation.

2. RQ-2: How does the labeling information of target project samples influence the performance of the UFR-OSFA approach?

To examine the impact of labeled data quantity, we perform experiments on the UFR-OSFA approach using target samples containing varying proportions of labeled information.

3. RQ-3: What is the contribution of each stage in the UFR-OSFA approach to the final prediction performance?

To further investigate its effectiveness in the HDP task, we analyze the stages of the UFR-OSFA approach.

4.2 | Datasets

In this paper, we collect 23 projects from five different datasets (AEEEM, ReLink, NASA, PROMISE, and SOFTLAB), which have been widely used in HDP research [14, 17, 42, 43]. These datasets are used in all our experiments. Table 3 provides detailed information about each dataset, including the dataset name, project name, the number of features, the sample count, the defective sample count, and the defect rate. The datasets are available at <https://drive.google.com/drive/folders/1vvTFdYLaoxoshWkZ8V6R7i6mm8MedMAG>.

TABLE 3 | Details of the dataset.

Dataset	Project	No. of features	No. of samples	No. of defective samples	Percentage of defective samples
AEEEM	EQ	61	324	129	39.81
	JDT	61	997	206	20.66
	LC	61	691	64	9.26
	ML	61	1862	245	13.16
	PDE	61	1497	209	13.96
ReLink	Apache	26	194	98	50.52
	Safe	26	56	22	39.29
	Zxing	26	399	118	29.57
NASA	KC1	21	2095	325	15.51
	MC1	38	8737	68	0.78
	PC1	37	735	61	8.30
	PC3	37	1099	138	12.56
	PC4	37	1379	178	12.91
PROMISE	camel-1.6	20	965	188	19.48
	lucene-2.4	20	340	203	59.71
	poi-3.0	20	442	281	63.57
	synapse-1.2	20	256	86	33.59
	velocity-1.6	20	229	78	34.06
SOFTLAB	AR1	29	121	9	7.44
	AR3	29	63	8	12.70
	AR4	29	107	20	18.69
	AR5	29	36	8	22.22
	AR6	29	101	15	14.85

4.3 | Baseline Models

To assess the efficacy of our approach, we compare it against several baseline methods. Concretely, four representative MP-HDP methods are chosen: CLSUP [19], MSMDA [20], CDAA [22], and DSKMDA [16]. Additionally, we include CTKCCA [39], which Chen et al. [13] identify as the state-of-the-art HDP method, and TDTSR [18], a recent HDP approach based on heterogeneous data preprocessing.

CTKCCA [39]: The cost-sensitive transfer kernel canonical correlation analysis method tackles the challenges of nonlinear inseparability and class imbalance in heterogeneous projects.

CLSUP [19]: A technique that utilizes cost-sensitive labeling and structure-consistent one-sided projection by integrating diverse source data with a limited set of target project data to tackle the class imbalance problem in HDP.

MSMDA [20]: Through multisource manifold discriminant alignment technology and a dual fuzzy privacy protection

algorithm based on sparse representation (SRDO), it resolves the data distribution differences and privacy protection issues in HDP. It is an MP-HDP method.

DSKMDA [16]: This method utilizes data sampling and kernel manifold discriminant alignment technology to tackle the challenges of nonlinear inseparability and class imbalance in MP-HDP settings.

TDTSR [18]: This method improves the effectiveness of training of the source dataset by selecting the appropriate source datasets and transferring data from the target dataset through correlation matching and data transfer methods.

CDAA [22]: By introducing a conditional domain adversarial adaptation network and using generative adversarial networks to solve the heterogeneity problem in HDP, this method improves the transfer performance between source and target projects through the combination of a generator, discriminator, and classifier. It is an MP-HDP method.

4.4 | Evaluation Metrics

In our study, we adopt multiple evaluation metrics—consistent with prior HDP research [14, 27, 29, 31, 39, 42, 44–46]—to comprehensively assess the performance of different methods. Specifically, we utilize the following metrics: AUC (Area Under the Curve), *F*-measure, MCC (Matthews Correlation Coefficient), and Popt (Percentage of Optimal Effort). AUC, *F*-measure, and MCC are three noneffort-aware performance metrics that are commonly used in HDP research to measure performance. Popt is an effort-aware metric, which implies whether it is possible to detect as many samples of defects as possible in the resource environment in the first place [47]. Higher values of these measures indicate better performance of the HDP model.

Before detailing these metrics, we introduce the confusion matrix, a fundamental tool for evaluating binary classification models. The confusion matrix comprises four components, which are presented in Table 4.

AUC measures the performance of a binary classification model by evaluating its ability to distinguish between positive and negative samples [42, 44]. Precisely, the ROC curve plots the True Positive Rate (TPR, $TPR = TP / (TP + FN)$) against the False Positive Rate (FPR, $FPR = FP / (FP + TN)$). The AUC value falls within the range of [0,1], where $AUC = 0.5$ signifies that the model's classification performance is similar to random guessing. $AUC = 1$ indicates that the model can distinguish between positive and negative classes perfectly. $AUC > 0.5$ indicates that the model is better than random guessing. Thus, the model's classification performance improves as the AUC value approaches 1.

***F*-measure** is a metric that combines Precision ($Precision = TP / (TP + FP)$) and Recall (i.e., TPR) [29, 45], which is calculated as

$$F\text{-measure} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (18)$$

The *F*-measure ranges from [0, 1], with higher values indicating better overall model performance. It provides a balanced assessment of Precision and Recall, making it particularly useful for imbalanced datasets.

MCC is a performance metric that considers all four classification outcomes: TP, TN, FP, and FN. It is widely used to evaluate binary classification models and is particularly effective for sample imbalance problems [43, 46]. MCC values range from [−1, 1] and are computed as follows:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (19)$$

TABLE 4 | Confusion matrix.

	Predict defective	Predict nondefective
Actual defective	True positive (TP)	False negative (FN)
Actual nondefective	False positive (FP)	True negative (TN)

Popt is a metric used in defect prediction scenarios. Here, code modules are ranked based on their predicted probability of being defective, allowing developers to prioritize testing and inspection of high-risk modules. Popt quantifies the number of defects detected after examining a specified percentage of the code. It is derived from the following two curves:

- The curve for random ordering indicates the progress of finding defects as testing proceeds, if modules are randomly selected.
- The curve for optimal ordering represents the progress of finding defects as testing proceeds, if we can perfectly predict the distribution of defects and start with the modules most likely to contain defects.

Popt's calculations are usually based on a defect discovery cumulative defect curve, which shows the cumulative percentage of defects found as the amount of code inspected increases. Popt's formula can be expressed as follows:

$$P_{opt}(m) = 1 - \Delta_{opt} = \frac{Area(optimal) - Area(m)}{Area(optimal) - Area(worst)} \quad (20)$$

where $Area(optimal)$, $Area(predicted)$, and $Area(worst)$ represent the areas under the curves for the best model, predicted model, and worst model, respectively. As with the previous HDP studies [14, 17, 48], this study assesses Popt at an effort level of 20%.

Essentially, we employ statistical methods to assess the significance of the results and determine whether performance differences across methods are statistically meaningful. Consistent with previous HDP studies [49–51], we initially apply the Wilcoxon signed-rank test, a nonparametric statistical test suitable for comparing differences between two sample groups, particularly for nonnormally distributed data. Typically, at a 95% confidence level, a *p*-value less than 0.05 in the Wilcoxon signed-rank test indicates a significant difference between the results of the two methods. Subsequently, we utilize Cliff's δ [52–55] to assess the magnitude of statistical differences between the results. Cliff's δ quantifies the degree of divergence between two sample groups and is well-suited for nonparametric comparisons. It ranges from −1 to 1, with values indicating the proportion by which one group's

TABLE 5 | Cliff's δ with effectiveness level.

Cliff's δ	Effectiveness level
$0.000 \leq \delta < 0.147$	Negligible
$0.147 \leq \delta < 0.330$	Small
$0.330 \leq \delta < 0.474$	Medium
$0.474 \leq \delta < 1.000$	Large

TABLE 6 | AUC and F-measure average results for UFR-OSFA and baseline methods.

Target	AUC										F-measure				
	UFR-OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	UFR-OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	
eq	0.742	0.642	0.673	0.552	0.683	0.687	0.710	0.592	0.550	0.508	0.583	0.588	0.557	0.528	
jdt	0.778	0.694	0.587	0.728	0.505	0.740	0.744	0.587	0.440	0.571	0.528	0.539	0.561	0.508	
lc	0.598	0.565	0.574	0.684	0.621	0.618	0.550	0.464	0.620	0.424	0.349	0.408	0.475	0.473	
ml	0.717	0.665	0.722	0.657	0.666	0.781	0.726	0.468	0.356	0.353	0.470	0.325	0.505	0.381	
pde	0.701	0.453	0.548	0.662	0.497	0.565	0.539	0.497	0.321	0.291	0.408	0.280	0.411	0.372	
Apache	0.779	0.734	0.696	0.579	0.659	0.672	0.736	0.606	0.526	0.550	0.564	0.514	0.605	0.514	
Safe	0.767	0.592	0.584	0.551	0.716	0.755	0.700	0.547	0.486	0.443	0.538	0.407	0.536	0.423	
Zxing	0.690	0.551	0.681	0.600	0.506	0.651	0.691	0.502	0.441	0.484	0.447	0.345	0.464	0.410	
KC1	0.655	0.645	0.515	0.584	0.465	0.550	0.643	0.544	0.479	0.342	0.362	0.397	0.435	0.400	
MC1	0.558	0.662	0.504	0.585	0.466	0.584	0.508	0.310	0.582	0.576	0.331	0.483	0.477	0.588	
PC1	0.642	0.788	0.575	0.653	0.571	0.666	0.631	0.509	0.582	0.342	0.325	0.327	0.510	0.470	
PC3	0.738	0.595	0.548	0.716	0.553	0.597	0.595	0.485	0.605	0.503	0.403	0.491	0.403	0.529	
PC4	0.808	0.698	0.714	0.742	0.689	0.808	0.721	0.588	0.464	0.444	0.387	0.396	0.589	0.415	
camel	0.664	0.696	0.650	0.568	0.591	0.569	0.649	0.512	0.492	0.500	0.380	0.448	0.463	0.530	
lucene	0.661	0.649	0.648	0.644	0.577	0.661	0.623	0.596	0.542	0.457	0.606	0.435	0.630	0.544	
poi	0.738	0.557	0.561	0.718	0.525	0.440	0.487	0.578	0.394	0.508	0.485	0.585	0.499	0.559	
synapse	0.719	0.713	0.545	0.661	0.498	0.727	0.654	0.674	0.610	0.522	0.554	0.477	0.576	0.474	
velocity	0.661	0.638	0.501	0.628	0.650	0.555	0.663	0.610	0.296	0.387	0.522	0.496	0.475	0.442	
ar1	0.737	0.502	0.515	0.525	0.731	0.601	0.509	0.462	0.494	0.607	0.519	0.567	0.229	0.625	
ar3	0.698	0.473	0.485	0.549	0.610	0.818	0.630	0.443	0.473	0.374	0.524	0.316	0.522	0.450	
ar4	0.740	0.487	0.497	0.689	0.515	0.575	0.570	0.606	0.417	0.590	0.533	0.535	0.459	0.490	
ar5	0.780	0.572	0.693	0.681	0.762	0.711	0.638	0.598	0.308	0.519	0.544	0.540	0.556	0.528	
ar6	0.669	0.540	0.544	0.604	0.581	0.611	0.628	0.447	0.270	0.355	0.442	0.321	0.434	0.393	
Average	0.706	0.614	0.590	0.633	0.593	0.650	0.632	0.532	0.467	0.463	0.470	0.444	0.494	0.480	
Improve	—	15.1%	19.8%	11.6%	19.1%	8.7%	11.7%	—	13.7%	14.8%	13.2%	19.6%	7.5%	10.7%	

(Continues)

TABLE 6 | (Continued)

Target	AUC										F-measure				
	UFR-					UFR-									
	OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	
Median	0.717	0.638	0.574	0.644	0.581	0.651	0.638	0.544	0.479	0.484	0.485	0.448	0.499	0.474	
Improve	—	12.5%	25.0%	11.4%	23.5%	10.2%	12.4%	—	13.6%	12.5%	12.1%	21.5%	9.1%	14.9%	
p-value	—	0.001	0.000	0.000	0.000	0.009	0.000	—	0.027	0.005	0.002	0.002	0.022	0.011	
Cliff's δ	—	0.622	0.739	0.588	0.698	0.365	0.569	—	0.357	0.422	0.395	0.527	0.274	0.391	

median exceeds the other. Table 5 outlines the effect size levels and their corresponding values. Finally, we visually analyze the indicator results using the Scott–Knott effect size difference (ESD) test [29, 56–59]. The Scott–Knott ESD test is a statistical method for comparing effect sizes among multiple groups, particularly effective for evaluating performance differences under various treatments or conditions. This method identifies significantly different groups by ranking the means of the groups.

4.5 | Experiment Settings

In the experiments, the network structure of the generator network is set to (unified feature dimension, 256), (256, 128), and (128, 64). The network structures for the source project scattering classifier and the target project clustering classifier are both (64, 32) and (1, 32). The model iterates 100 times, with ReLU activation functions in the hidden layers and a Sigmoid activation function in the output layer. Following the settings of Qin et al. [35], the loss function hyperparameters α , β , and λ are set to 0.75, 0.1, and 0.1, respectively. The parameter settings for other baseline methods are aligned with those in the original papers.

To ensure fair comparison and prevent data leakage, we follow the standard MP-HDP setting [16, 17], where a certain amount of labeled data from the target project is available to assist with training. Specifically, for each target project, the dataset is randomly divided into three mutually exclusive subsets: 10% labeled data from the target project (X_L) used for supervised learning, 10% unlabeled samples (X_U^{train}) used only in our method for domain adaptation, 80% samples (X_U^{test}), reserved strictly for testing. All MP-HDP methods and our proposed UFR-OSFA, are trained using the entire source project data (X_S) and exactly the same 10% labeled target data (X_L). And they all use 80% unlabeled data (X_U^{test}) for testing. This ensures a fair comparison, as each model receives the same amount of labeled supervision from the target project. The only difference is that UFR-OSFA additionally exploits X_U^{train} through entropy-based semi-supervised learning, while X_U^{train} remains invisible to other purely supervised MP-HDP baselines; the additional provision of unlabeled samples does not change their parameter updates (there are no entropic terms in other baselines). This experimental setup is likewise common in some classic semi-supervised domain adaptation papers. In the discussion section, we analyze the impact of X_U^{train} on performance.

5 | Experiments Results

5.1 | Comparison of Results of UFR-OSFA With the Baseline Methods

[Experiment Goal]: This research question evaluates the comparative effectiveness of the proposed UFR-OSFA against existing baseline methods.

[Experiment Design]: As detailed in Table 3, our experimental design involves selecting one project from the dataset as the target project while sequentially choosing source projects from other datasets. For example, if we chose KC1 from the NASA dataset as the target project, the source projects are selected

TABLE 7 | MCC and Popt average results for UFR-OSFA and baseline methods.

Target	MCC							Popt						
	UFR-OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	UFR-OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA
eq	0.265	0.206	0.180	0.187	0.192	0.183	0.175	0.431	0.559	0.445	0.554	0.458	0.418	0.361
jdt	0.375	0.316	0.165	0.201	0.248	0.321	0.208	0.523	0.504	0.462	0.440	0.426	0.438	0.429
lc	0.195	0.158	0.173	0.235	0.155	0.156	0.219	0.478	0.514	0.411	0.496	0.525	0.580	0.442
ml	0.342	0.285	0.169	0.225	0.177	0.277	0.161	0.513	0.432	0.392	0.389	0.478	0.428	0.455
pde	0.202	0.123	0.172	0.175	0.161	0.211	0.158	0.483	0.413	0.278	0.296	0.338	0.379	0.370
Apache	0.262	0.216	0.236	0.270	0.188	0.205	0.167	0.529	0.455	0.504	0.426	0.415	0.508	0.419
Safe	0.303	0.250	0.217	0.285	0.234	0.266	0.192	0.415	0.364	0.303	0.351	0.459	0.296	0.310
Zxing	0.272	0.251	0.177	0.255	0.214	0.207	0.186	0.488	0.462	0.481	0.379	0.441	0.412	0.351
KC1	0.212	0.199	0.156	0.201	0.183	0.169	0.164	0.574	0.256	0.359	0.421	0.354	0.471	0.473
MC1	0.278	0.135	0.161	0.176	0.127	0.307	0.272	0.488	0.150	0.271	0.232	0.310	0.352	0.464
PC1	0.293	0.213	0.202	0.231	0.180	0.239	0.163	0.427	0.400	0.388	0.363	0.326	0.438	0.451
PC3	0.283	0.234	0.121	0.212	0.199	0.214	0.181	0.532	0.279	0.400	0.453	0.388	0.373	0.510
PC4	0.325	0.316	0.205	0.209	0.162	0.232	0.160	0.499	0.284	0.412	0.518	0.498	0.521	0.447
ant	0.291	0.223	0.168	0.209	0.244	0.218	0.225	0.487	0.253	0.241	0.398	0.349	0.381	0.481
camel	0.296	0.220	0.252	0.267	0.215	0.237	0.190	0.447	0.492	0.418	0.525	0.439	0.449	0.504
poi	0.314	0.227	0.191	0.212	0.220	0.271	0.259	0.464	0.556	0.417	0.450	0.420	0.396	0.513
velocity	0.308	0.318	0.225	0.267	0.184	0.272	0.167	0.457	0.419	0.273	0.399	0.445	0.426	0.390
xalan	0.200	0.092	0.172	0.216	0.150	0.203	0.138	0.458	0.527	0.307	0.528	0.409	0.422	0.307
ar1	0.258	0.196	0.188	0.275	0.173	0.264	0.275	0.329	0.332	0.172	0.461	0.187	0.446	0.316
ar3	0.287	0.196	0.201	0.233	0.156	0.261	0.144	0.403	0.383	0.351	0.215	0.354	0.256	0.251
ar4	0.296	0.212	0.175	0.265	0.212	0.289	0.187	0.383	0.276	0.392	0.379	0.292	0.236	0.276
ar5	0.266	0.182	0.207	0.260	0.223	0.302	0.202	0.288	0.275	0.270	0.156	0.227	0.293	0.275
ar6	0.260	0.192	0.168	0.248	0.178	0.199	0.159	0.468	0.240	0.394	0.391	0.345	0.433	0.404
Average	0.277	0.216	0.186	0.231	0.190	0.239	0.189	0.459	0.384	0.363	0.401	0.386	0.407	0.400
Improve	—	28.7%	49.0%	20.1%	45.8%	16.0%	46.6%	—	19.8%	26.7%	14.6%	19.0%	13.0%	14.8%

(Continues)

TABLE 7 | (Continued)

Target	MCC										Popt				
	UFR-					UFR-									
	OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	OSFA	CTKCCA	CLUSP	DSKMDA	MSMDA	TDTSR	CDA	
Median	0.283	0.213	0.177	0.231	0.184	0.237	0.181	0.468	0.400	0.392	0.399	0.409	0.422	0.419	
Improve	—	32.7%	59.4%	22.4%	53.6%	19.3%	56.2%	—	17.1%	19.5%	17.2%	14.5%	10.9%	11.7%	
<i>p</i> -value	—	0.000	0.000	0.001	0.000	0.000	0.000	—	0.018	0.000	0.017	0.000	0.004	0.000	
Cliff's δ	—	0.573	0.898	0.595	0.879	0.429	0.856	—	0.391	0.671	0.388	0.527	0.448	0.440	

sequentially from the 18 projects across the AEEEM, ReLink, PROMISE, and SOFTLAB datasets. Thus, there are 405 ($5 \times 18 + 3 \times 15 + 5 \times 18 + 5 \times 18 + 5 \times 18$) unique source-target project combinations in total. Each combination runs 30 times, with average performance metrics calculated across the repetitions.

[Experiment Result]: Tables 6 and 7 summarize the AUC, *F*-measure, MCC, and Popt prediction results for our approach compared to the baseline methods across 23 target projects. The average, median, Wilcoxon signed-rank test results, and Cliff's δ effect sizes for each of these methods are also provided. The best values for each target project across the four evaluation metrics are highlighted in bold.

In general, the results demonstrate that UFR-OSFA consistently achieves superior or comparable performance in AUC, *F*-measure, MCC, and Popt compared to the six baseline methods, confirming its effectiveness for HDP. Regarding AUC, our approach achieves the best predictive values in 13 projects, showing improvements in the average AUC ranging from 8.7% to 19.8%, and in the median AUC from 10.2% to 25%. For *F*-measure, our approach achieves the best values in 12 projects, showing improvements ranging from 7.5% to 14.8% and in the median *F*-measure from 9.1% to 21.5%. Similarly, for MCC, the best predictive values are achieved in 15 projects, with improvements in the average MCC from 16.0% to 49% and in the median MCC from 19.3% to 59.4%. Finally, for Popt, our approach achieves the best values in 12 projects, showing improvement ranges for average Popt from 13% to 26.7% and for median Popt from 11.7% to 19.5%.

Statistical analysis further confirmed the significant advantages of UFR-OSFA. Wilcoxon signed-rank tests yield *p*-values below 0.05 for all metrics, indicating statistically significant performance differences compared to each baseline method. Moreover, Cliff's δ analysis revealed that UFR-OSFA exhibited a medium to large effect size compared to other methods, providing further evidence of the substantial enhancement in performance of UFR-OSFA. These improvements are not only numerical but also hold practical significance in application.

To clearly illustrate the performance results of all methods across all datasets, we visualize the results using box plots, as shown in Figure 2. The mean values are also marked in the figures with red asterisks. These figures show that our approach produces superior performance outcomes in both effort-aware and noneffort-aware metrics compared with the baseline methods. In summary, UFR-OSFA achieves better or comparable results across the four performance metrics compared to the baseline methods. Additionally, Figure 3 presents the Scott-Knott ESD test results for *F*-measure and AUC across all projects, comparing the UFR-OSFA with the HDP baseline methods. The *x*-axis represents the compared methods, while the *y*-axis shows the performance ranking values. A vertical line indicates each method's ranking range across all projects. The dots on the lines represent the average rankings and different colors, denoting statistically distinct groups. A smaller ranking value means better performance. As shown in Figure 3, it is clear that our approach consistently achieves the lowest average ranking across all four evaluation metrics and is assigned to a group that does not include any other HDP methods. This confirms that our approach is statistically more effective than the different methods.

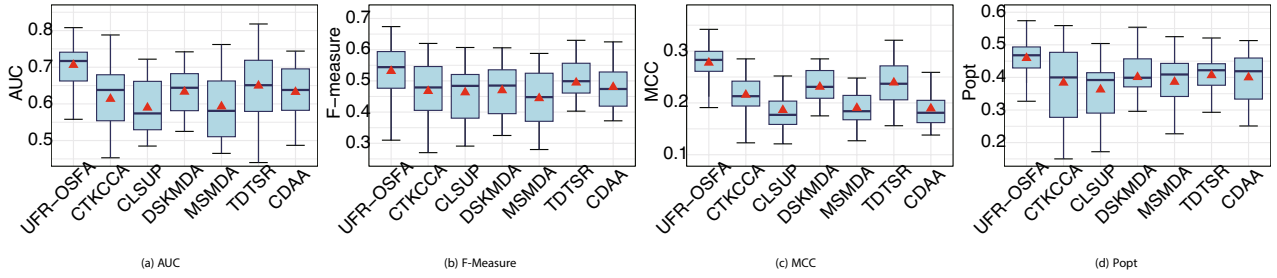


FIGURE 2 | Box plots of AUC, F -measure, MCC, and Popt for UFR-OSFA and baseline methods.

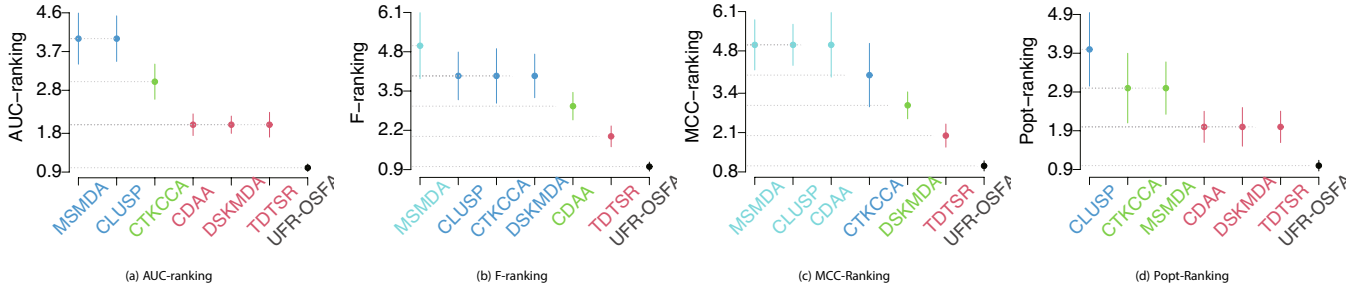


FIGURE 3 | Scott-Knott ESD test results for AUC, F -measure, MCC, and Popt for UFR-OSFA and baseline methods.

The superior performance and stability of UFR-OSFA in HDP can be attributed to two key factors. First, we unify the features of different projects based on feature distributional similarity calculations and work to minimize data distribution discrepancies, effectively transforming the problem of heterogeneous domain adaptation into one of homogeneous domain adaptation. Second, we address the issues of conditional distribution mismatch and decision boundary bias toward the source project through oppositional optimization of source project scattering and target project clustering. Through oppositional training, we effectively align the heterogeneous-domain feature distributions while simultaneously improving the target project classification performance by leveraging both labeled and unlabeled data.

5.2 | The Impact of Labeled Target Project on UFR-OSFA

[Experiment Goal]: To assess the validity of the sample labeling information in the target projects for our approach and assess the effect of labeled sample quantity, we perform experiments using different proportions of labeled target projects.

[Experiment Design]: Specifically, the percentage of training target data X_L ranges from 0% to 50%, with a step size of 10. For each percentage, we run the UFR-OSFA method 30 times. Keeping all other experimental settings constant, we finally report the results of AUC, F -measure, MCC, and Popt.

[Experiment Result]:

Table 8 and Table 9 present the performance results of the UFR-OSFA approach using varying proportions of labeled target samples across four evaluation metrics. Figure 4 provides a

more intuitive visualization of the average results for the UFR-OSFA approach under different proportions of labeled target samples. As the proportion of labeled samples in the target project increases, a noticeable trend of improved model predictive performance indicates that a more significant number of labeled samples enhances the model's effectiveness. Specifically, below 10%, the UFR-OSFA approach demonstrates substantial performance improvements, the UFR-OSFA using 10% of labeled samples from the target project, achieves an average improvement of 21.7% in F -measure, 27.0% in AUC, 34.0% in MCC, and 20.2% in Popt compared to not using any labeled samples from the target project (0%). Furthermore, Wilcoxon signed-rank statistical tests confirm the differences between the results at this level.

The significant performance improvement is mainly due to the precise supervision provided by the labeled target project data, which allows the model to learn the class distribution of the target project better. When the model receives labeled data from the target project, it can more accurately adjust the output of the feature generator based on these explicit labels, leading to tighter clustering of the target project samples within their respective class regions. This enhances the alignment between the target and source project features. Additionally, the labeled target project data helps reduce the generalization error caused by over-reliance on source project samples, especially in cases where there is a substantial distribution difference between the source and target projects. This mixed-project SDP approach, which leverages labeled data, makes UFR-OSFA more robust in the target project, particularly in scenarios with limited labeled samples. The model can better adjust to the target project's feature distribution using this supervised learning process. This enhances classification performance and improves the generalization ability for heterogeneous-domain transfer.

TABLE 8 | AUC and *F*-measure results for different proportions of labeled target project samples.

Target	AUC						<i>F</i> -measure					
	10%	0%	20%	30%	40%	50%	10%	0%	20%	30%	40%	50%
eq	0.742	0.787	0.745	0.794	0.725	0.692	0.592	0.508	0.612	0.595	0.606	0.620
jdt	0.778	0.582	0.731	0.621	0.685	0.706	0.587	0.388	0.525	0.634	0.491	0.534
lc	0.598	0.566	0.767	0.799	0.513	0.744	0.464	0.391	0.484	0.614	0.573	0.626
ml	0.717	0.592	0.832	0.720	0.737	0.796	0.468	0.416	0.598	0.556	0.442	0.476
pde	0.701	0.659	0.747	0.696	0.755	0.695	0.497	0.423	0.529	0.431	0.560	0.456
Apache	0.779	0.565	0.739	0.671	0.730	0.764	0.606	0.382	0.521	0.638	0.556	0.487
Safe	0.767	0.579	0.868	0.705	0.691	0.635	0.547	0.415	0.589	0.367	0.600	0.526
Zxing	0.690	0.562	0.678	0.516	0.722	0.788	0.502	0.360	0.555	0.540	0.619	0.494
KC1	0.655	0.551	0.632	0.744	0.778	0.772	0.544	0.421	0.529	0.470	0.573	0.667
MC1	0.558	0.616	0.765	0.666	0.718	0.783	0.310	0.306	0.428	0.591	0.605	0.631
PC1	0.642	0.547	0.595	0.807	0.669	0.738	0.509	0.530	0.452	0.623	0.556	0.554
PC3	0.738	0.556	0.507	0.666	0.744	0.679	0.485	0.529	0.448	0.472	0.602	0.430
PC4	0.808	0.535	0.798	0.812	0.855	0.818	0.588	0.391	0.530	0.307	0.305	0.634
ant	0.664	0.512	0.654	0.828	0.720	0.734	0.512	0.495	0.479	0.555	0.563	0.641
camel	0.661	0.472	0.742	0.766	0.822	0.721	0.596	0.360	0.551	0.490	0.487	0.620
poi	0.738	0.512	0.697	0.731	0.687	0.765	0.578	0.264	0.548	0.572	0.614	0.512
velocity	0.719	0.520	0.724	0.780	0.682	0.820	0.674	0.446	0.666	0.593	0.450	0.731
xalan	0.661	0.536	0.787	0.795	0.723	0.705	0.610	0.454	0.582	0.610	0.555	0.456
ar1	0.737	0.711	0.645	0.694	0.841	0.718	0.462	0.517	0.474	0.656	0.632	0.538
ar3	0.698	0.604	0.635	0.588	0.676	0.799	0.443	0.424	0.454	0.554	0.573	0.579
ar4	0.740	0.545	0.636	0.633	0.698	0.627	0.606	0.436	0.534	0.656	0.474	0.614
ar5	0.780	0.618	0.677	0.786	0.787	0.720	0.598	0.371	0.603	0.484	0.557	0.554
ar6	0.669	0.623	0.712	0.637	0.785	0.759	0.447	0.402	0.498	0.544	0.640	0.427
Average	0.706	0.580	0.709	0.715	0.728	0.738	0.532	0.419	0.534	0.546	0.549	0.557
<i>p</i> -value	—	0.000	0.915	0.796	0.242	0.110	—	0.000	0.595	0.513	0.403	0.354

When the proportion of labeled samples exceeds 10%, the performance improvement begins to level off. Especially, increasing the amount of labeled data (e.g., 30%, 40%, and 50%) results in only marginal gains compared to the 10% labeled. Statistical analysis shows that the *p*-values for all these proportions are above the 0.05 threshold, indicating that the differences are not statistically significant. This suggests the presence of a point of diminishing returns, beyond which adding more labeled samples does not lead to significant performance improvements. This phenomenon highlights two important points. First, the UFR-OSFA method demonstrates strong robustness with relatively few labeled target samples, confirming its efficiency in improving performance with limited labeled data. Second, the diminishing returns imply that beyond a certain threshold, the model has already learned most of the key class distributions and feature representations necessary for accurate classification, and additional labeled data do not significantly enhance predictive capability.

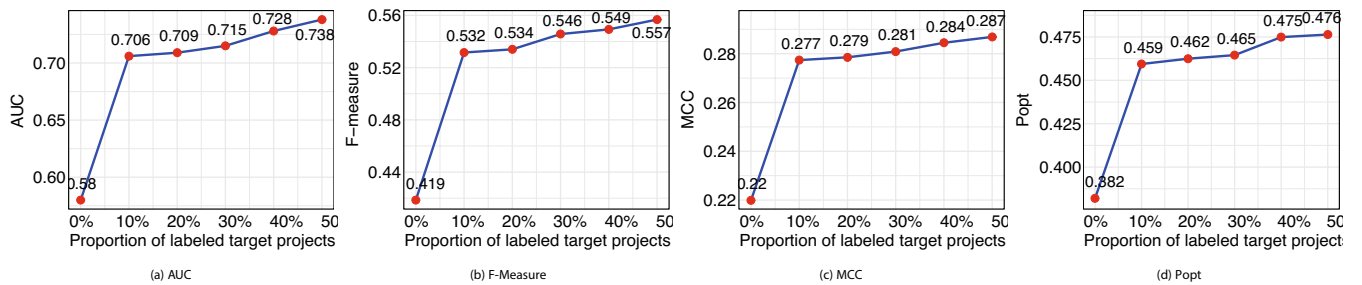
Consequently, we utilize 10% of the labeled target project in our experiments.

Although Table 8 and Table 9 confirm that adding 10% labeled target samples almost always helps, some projects exhibit at least one metric that deteriorates. For example, EQ and MC1 show a modest AUC drop (−0.045 and −0.058, respectively), while AR1 loses 0.055 *F*-measure points (and a similar amount in MCC for MC1) even though their AUC continues to rise. A closer look at the dataset statistics explains these outliers.

1. Extreme class ratios: MC1 contains only 0.78% defective modules (68/8737). When only 10% of the files are labeled, the classifier receives roughly 13 positive and 1730 negative labeled samples. Such extreme skew amplifies the influence of any mis-labeled positive, causing the supervised branch to overfit and shift the ranking order of the many negative modules, hence a lower AUC.

TABLE 9 | MCC and Popt results for different proportions of labeled target project samples.

Target	MCC						Popt					
	10%	0%	20%	30%	40%	50%	10%	0%	20%	30%	40%	50%
eq	0.265	0.217	0.322	0.337	0.259	0.303	0.431	0.337	0.490	0.427	0.463	0.504
jdt	0.375	0.236	0.332	0.364	0.362	0.340	0.523	0.403	0.529	0.536	0.489	0.528
lc	0.195	0.150	0.222	0.198	0.214	0.198	0.478	0.428	0.501	0.515	0.548	0.467
ml	0.342	0.290	0.319	0.335	0.268	0.323	0.513	0.492	0.493	0.457	0.517	0.460
pde	0.202	0.198	0.225	0.290	0.277	0.415	0.483	0.432	0.463	0.459	0.475	0.510
Apache	0.262	0.188	0.298	0.343	0.267	0.342	0.529	0.243	0.496	0.513	0.537	0.529
Safe	0.303	0.202	0.299	0.299	0.273	0.251	0.415	0.391	0.404	0.416	0.445	0.589
Zxing	0.272	0.244	0.266	0.353	0.309	0.296	0.488	0.294	0.491	0.469	0.511	0.644
KC1	0.212	0.234	0.232	0.242	0.247	0.251	0.574	0.493	0.527	0.611	0.653	0.545
MC1	0.278	0.296	0.248	0.209	0.327	0.300	0.488	0.497	0.499	0.446	0.479	0.470
PC1	0.293	0.276	0.282	0.321	0.365	0.220	0.427	0.366	0.427	0.442	0.450	0.403
PC3	0.283	0.257	0.242	0.251	0.276	0.323	0.532	0.407	0.556	0.502	0.598	0.484
PC4	0.325	0.217	0.335	0.239	0.288	0.324	0.499	0.282	0.509	0.457	0.445	0.459
ant	0.291	0.148	0.266	0.240	0.315	0.233	0.487	0.299	0.481	0.576	0.558	0.464
camel	0.296	0.195	0.299	0.255	0.301	0.322	0.447	0.423	0.414	0.467	0.456	0.390
poi	0.314	0.192	0.287	0.258	0.161	0.240	0.464	0.342	0.453	0.523	0.454	0.530
velocity	0.308	0.233	0.312	0.240	0.293	0.337	0.457	0.343	0.488	0.522	0.433	0.418
xalan	0.200	0.201	0.222	0.240	0.218	0.262	0.458	0.427	0.454	0.429	0.572	0.509
ar1	0.258	0.218	0.262	0.228	0.363	0.291	0.329	0.463	0.357	0.263	0.306	0.399
ar3	0.287	0.274	0.284	0.370	0.278	0.261	0.403	0.529	0.419	0.458	0.397	0.268
ar4	0.296	0.154	0.286	0.278	0.291	0.258	0.383	0.274	0.401	0.406	0.375	0.529
ar5	0.266	0.220	0.274	0.290	0.291	0.250	0.288	0.206	0.299	0.286	0.265	0.310
ar6	0.260	0.219	0.292	0.281	0.301	0.258	0.468	0.414	0.485	0.502	0.496	0.548
Average	0.277	0.220	0.279	0.281	0.284	0.287	0.459	0.382	0.462	0.465	0.475	0.476
<i>p</i> -value	—	0.001	0.939	0.867	0.308	0.616	—	0.002	0.533	0.704	0.207	0.420

**FIGURE 4** | Impact of different proportions of labeled target samples on performance.

2. Small target projects: EQUATION (324 samples) and AR1 (121 samples) are the two smaller target projects. Limited labeled data means that the entropy-minimization step may overfit to whichever modules happen to be labeled, hurting recall and hence *F*-measure.
3. Limited or sparse feature overlap: After metric unification, some target projects (such as EQ) retain fewer matched features, reducing the signal available to the adversarial optimizer and hurting ranking quality.

TABLE 10 | AUC, F-measure, MCC, and Popt performance results of different phases.

Target	AUC			F-measure			MCC			Popt		
	BASE	Phase I	Phase II	BASE	Phase I	Phase II	BASE	Phase I	Phase II	BASE	Phase I	Phase II
eq	0.480	0.515	0.742	0.229	0.467	0.592	0.089	0.185	0.265	0.309	0.353	0.431
jdt	0.556	0.596	0.778	0.320	0.447	0.587	0.287	0.286	0.375	0.333	0.413	0.523
lc	0.124	0.330	0.598	0.037	0.135	0.464	0.081	0.139	0.195	0.159	0.294	0.478
ml	0.499	0.543	0.717	0.209	0.291	0.468	0.176	0.158	0.342	0.210	0.249	0.513
pde	0.361	0.432	0.701	0.152	0.264	0.497	0.083	0.121	0.202	0.155	0.220	0.483
Apache	0.578	0.614	0.779	0.315	0.459	0.606	0.335	0.148	0.262	0.321	0.423	0.529
Safe	0.495	0.570	0.767	0.231	0.323	0.547	0.119	0.207	0.303	0.241	0.290	0.415
Zxing	0.478	0.565	0.690	0.176	0.338	0.502	0.142	0.153	0.272	0.201	0.297	0.488
KC1	0.442	0.459	0.655	0.108	0.315	0.544	0.066	0.102	0.212	0.141	0.278	0.574
MC1	0.325	0.390	0.558	0.076	0.217	0.310	0.020	0.211	0.278	0.183	0.233	0.488
PC1	0.436	0.499	0.642	0.201	0.238	0.509	0.024	0.138	0.293	0.106	0.195	0.427
PC3	0.392	0.535	0.738	0.119	0.155	0.485	0.052	0.078	0.283	0.035	0.141	0.532
PC4	0.511	0.627	0.808	0.182	0.324	0.588	0.189	0.169	0.325	0.105	0.313	0.499
camel	0.450	0.464	0.664	0.115	0.178	0.512	0.010	0.077	0.291	0.044	0.166	0.487
lucene	0.440	0.526	0.661	0.277	0.519	0.596	0.129	0.162	0.296	0.349	0.388	0.447
poi	0.327	0.580	0.738	0.162	0.512	0.578	0.120	0.229	0.314	0.223	0.414	0.464
synapse	0.450	0.476	0.719	0.213	0.389	0.674	0.043	0.132	0.308	0.254	0.353	0.457
velocity	0.447	0.497	0.661	0.171	0.370	0.610	0.118	0.102	0.200	0.198	0.330	0.458
ar1	0.350	0.477	0.737	0.161	0.195	0.462	0.079	0.080	0.258	0.096	0.166	0.329
ar3	0.439	0.604	0.698	0.083	0.348	0.443	0.052	0.229	0.287	0.093	0.347	0.403
ar4	0.385	0.555	0.740	0.234	0.408	0.606	0.112	0.147	0.296	0.142	0.372	0.383
ar5	0.439	0.638	0.780	0.156	0.475	0.598	0.073	0.210	0.266	0.155	0.360	0.288
ar6	0.435	0.542	0.669	0.147	0.292	0.447	0.099	0.186	0.260	0.181	0.276	0.468
Average	0.428	0.523	0.706	0.177	0.333	0.532	0.109	0.159	0.277	0.184	0.299	0.459

Overall, these findings suggest that projects with either an extreme defect ratio or very few modules may need a slightly larger target-label budget to stabilize performance.

5.3 | Performance of the UFR-OSFA at Various Stages

[Experiment Goal]: In the previous research question, we verify the significance of the performance of the UFR-OSFA approach on HDP. To further investigate its effectiveness in the HDP task, we evaluate the UFR-OSFA approach in all phases.

[Experiment Design]: The UFR-OSFA approach first addresses the issue of inconsistent feature quantities by unifying the features. Then, it reduces data distribution differences through an oppositional-structure entropy optimization mechanism. To evaluate the impact of these two stages on the HDP model, we establish a baseline method (BASE), which aligns the features by randomly selecting features between the source and target projects and then constructs the HDP model using a random forest classifier. Furthermore, Phase I refers to constructing the HDP model using a random forest classifier after the unified feature representation is completed, which shares some similarities with the TDTSR method [18]. By comparing the performance of the BASE method and Phase I across four evaluation metrics, we can analyze the effectiveness of the strategy of unified feature representation and its impact on the final predictive performance. Phase II, which is the proposed UFR-OSFA approach in this paper, further introduces a contrasting entropy optimization mechanism based on Phase I. To be exact, the source project scattering classifier ensures the model avoids overfitting on the source project, while the target project's clustering classifier ensures accurate classification of the target project samples, addressing data distribution differences and achieving feature alignment between the source and target domains. By comparing the predictive performance of Phase I and Phase II, the effectiveness of each stage and its contribution to the final predictive results can be evaluated.

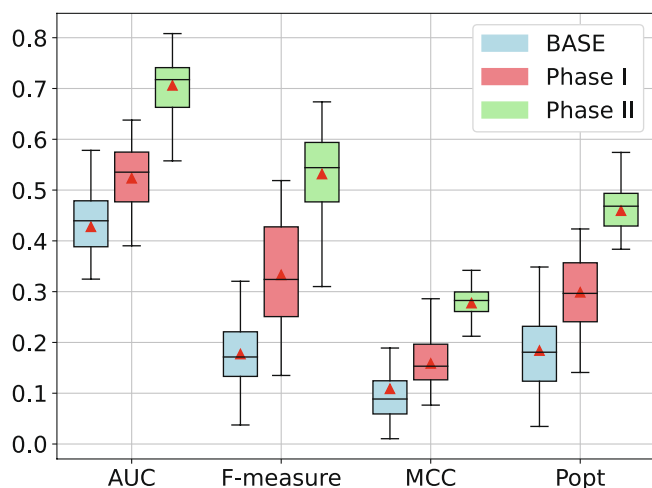


FIGURE 5 | AUC, F -measure, MCC, and Popt performance results of different phase.

[Experiment Result]: Table 10 presents the performance results of the BASE method, Phase I, and Phase II across 4 evaluation metrics: AUC, F -measure, MCC, and Popt. Figure 5 provides boxplot representations of the prediction results for the BASE method, Phase I, and Phase II under these four metrics. In Figure 5, it is evident that Phase II achieved the best performance, followed by Phase I, with the BASE method performing the worst. Specifically, across the four evaluation metrics, Phase I showed a significant improvement in predictive performance compared to the BASE method, with an average increase in AUC of 22.3%, an average increase in the F -measure of 88.0%, an average increase in MCC of 45.6%, and an average increase in Popt of 45.6%. These results highlight the effectiveness of the unified feature representation process, which helps reduce distribution differences between source and target projects. It indicates that feature unification plays a crucial role in reducing distribution discrepancies and improving prediction accuracy. Furthermore, it can be observed that Phase II further improves prediction performance compared to Phase I, with an average increase in AUC of 35.0%, an average increase in the F -measure of 59.7%, an average increase in MCC of 75.1%, and an average increase in Popt of 53.8%. This demonstrates the effectiveness of our approach to OSFA.

6 | Discussion

6.1 | Training and Prediction Time Comparisons

In this section, we investigate the training prediction time using different methods. The project pair camel-1.6 to KC1 (camel-1.6 is the source project, KC1 is the target project) is taken as an example. Table 11 shows the average training and prediction times (in seconds) for UFR-OSFA and baseline methods on this project pair, with 10% labeled target data.

UFR-OSFA exhibits a moderate increase in training time compared to CTKCCA, CLUSP, DSKMDA, MSMDA, and TDTSR, primarily due to the iterative training of the generator and dual classifiers in the OSFA stage. CDAA, which employs a generative adversarial network, has the highest training time due to its complex discriminator training. Prediction times for UFR-OSFA are slightly higher than those of CTKCCA, CLUSP, DSKMDA, MSMDA, and TDTSR but lower than CDAA, reflecting the efficiency of the single forward pass through the generator and target classifier. Despite

TABLE 11 | Training and prediction time comparisons (seconds).

Method	Training time	Prediction time
UFR-OSFA	12.4	0.82
CTKCCA	10.2	0.67
CLUSP	9.8	0.65
DSKMDA	11.5	0.73
MSMDA	11.3	0.71
TDTSR	8.9	0.60
CDAA	14.8	0.95

the increased computational cost, UFR-OSFA's superior performance (as shown in Section 4.1) justifies the tradeoff, particularly for applications where prediction accuracy is critical.

6.2 | Effect of Removing Target Clustering Entropy

In the default experimental setting of this study, UFR-OSFA is trained not only with 10% of the labeled target samples but also with an equal proportion of unlabeled target samples X_U^{train} .

Through the target-side conditional entropy term H_t , these unlabeled samples are implicitly “pulled” toward their potential class centers, enabling the generator to produce features that quickly cluster within the target domain while remaining smoothly aligned with the source domain. To quantify the true contribution of this unlabeled information, we create a stripped-down variant—UFR-OSFA-woU—that simply removes X_U^{train} , consequently, eliminates the H_t term. Aside from this change, UFR-OSFA-woU retains exactly the same network architecture, hyperparameter settings, the 10% labeled target data X_L , and all

TABLE 12 | AUC, F -measure, MCC, and Popt performance results of UFR-OSFA and UFR-OSFA-woU.

Target	AUC		F -measure		MCC		Popt	
	UFR-OSFA	UFR-OSFA-woU	UFR-OSFA	UFR-OSFA-woU	UFR-OSFA	UFR-OSFA-woU	UFR-OSFA	UFR-OSFA-woU
eq	0.742	0.680	0.592	0.513	0.265	0.222	0.431	0.399
jdt	0.778	0.688	0.587	0.561	0.375	0.318	0.523	0.433
lc	0.598	0.591	0.464	0.458	0.195	0.193	0.478	0.443
ml	0.717	0.633	0.468	0.449	0.342	0.301	0.513	0.471
pde	0.701	0.667	0.497	0.462	0.202	0.181	0.483	0.453
Apache	0.779	0.743	0.606	0.574	0.262	0.263	0.529	0.497
Safe	0.767	0.754	0.547	0.516	0.303	0.278	0.415	0.425
Zxing	0.690	0.652	0.502	0.472	0.272	0.198	0.488	0.451
KC1	0.655	0.619	0.544	0.540	0.212	0.182	0.574	0.536
MC1	0.558	0.519	0.310	0.305	0.278	0.241	0.488	0.455
PC1	0.642	0.624	0.509	0.474	0.293	0.248	0.427	0.375
PC3	0.738	0.678	0.485	0.470	0.283	0.273	0.532	0.489
PC4	0.808	0.738	0.588	0.567	0.325	0.261	0.499	0.499
camel	0.664	0.638	0.512	0.505	0.291	0.292	0.487	0.406
lucene	0.661	0.625	0.596	0.574	0.296	0.289	0.447	0.417
poi	0.738	0.686	0.578	0.542	0.314	0.291	0.464	0.428
synapse	0.719	0.687	0.674	0.646	0.308	0.301	0.457	0.400
velocity	0.661	0.641	0.610	0.588	0.200	0.193	0.458	0.369
ar1	0.737	0.687	0.462	0.457	0.258	0.220	0.329	0.352
ar3	0.698	0.694	0.443	0.424	0.287	0.269	0.403	0.381
ar4	0.740	0.698	0.606	0.587	0.296	0.301	0.383	0.318
ar5	0.780	0.754	0.598	0.577	0.266	0.263	0.288	0.276
ar6	0.669	0.629	0.447	0.431	0.260	0.222	0.468	0.373
Average	0.706	0.666	0.532	0.508	0.277	0.252	0.459	0.420
Improve	—	6.0%	—	4.6%	—	10.0%	—	9.5%
Median	0.717	0.678	0.544	0.513	0.283	0.263	0.468	0.425
Improve	—	5.8%	—	6.1%	—	7.6%	—	10.1%
p -value	—	0.001	—	0.014	—	0.002	—	0.003
Cliff's δ	—	0.391	—	0.233	—	0.312	—	0.391

source-project data used in the original model. Following the original protocol, we run 30 randomized trials on 23 publicly available projects and record four evaluation metrics: AUC, F -measure, MCC, and Popt.

Table 12 presents the experimental results of UFR-OSFA and UFR-OSFA-woU. The results indicate that after the removal of unlabeled samples, the model's performance declined across multiple metrics: the average AUC dropped from 0.706 to 0.666 (-6.0%), the F -measure decreased from 0.532 to 0.508 (-4.6%), the MCC exhibited a notable decline from 0.277 to 0.252 (-9.9%), and the Popt metric fell from 0.459 to 0.420 (-9.5%). Wilcoxon signed-rank tests yielded $p < 0.05$ for all four metrics, and Cliff's δ ranged from 0.23 to 0.4, indicating statistically significant degradations with small to medium effect sizes.

These degradations can be attributed to several key factors. First, with H_t removed, the generator no longer receives the cue that pulls target features together, so intraclass compactness deteriorates and the decision boundary must rely more heavily on the limited X_L . Second, lacking X_U^{train} to estimate the target conditional distribution, the alignment becomes less precise; the decision boundary leans more heavily on the sparse X_L , increasing the source-target dispersion gap and causing AUC and Popt to fall in tandem. Nevertheless, UFR-OSFA-woU maintained an advantage over purely supervised baselines such as CDAA, DSKMDA, and MP-HDP in most projects. This can be attributed to its unified feature representation and adversarial entropy optimization framework, which retains the source-side entropy dispersion H_s and cross-entropy terms jointly supervised by source and target labels. In summary, the inclusion of just 10% unlabeled target samples plays a crucial role in improving cross-domain generalization. The entropy-driven clustering they enable not only substantially improves accuracy and stability but also mitigates model overfitting of decision boundaries to source-specific biases in small-sample settings.

7 | Threats to Validity

As with most HDP studies, we discuss several potential threats to the validity of our approach, including internal threats, external threats, and construct threats.

7.1 | Internal Validity

First, concerning the baseline methods, we meticulously reproduce them based on their original literature descriptions. Although we rigorously verify the code, baselines, and datasets, undetected errors may remain. Second, for hyperparameter tuning, we employ a random grid search to identify near-optimal configurations. However, exhaustive testing of all combinations is infeasible, and some parameters are set empirically. Future work will focus on further optimizing these parameters.

7.2 | External Validity

To ensure a fair comparison between our approach and baseline methods, we carry out experiments on 23 projects in five

datasets: AEEEM, ReLink, NASA, PROMISE, and SOFTLAB. Researchers in the SDP field have widely used these datasets, and the projects exhibit variation in their features and sizes, further validating our approach's generalizability. However, because all of these projects come from public datasets, it is still necessary to evaluate our approach to a wider range of projects in the future.

7.3 | Construct Validity

To mitigate construct threats, we select three noneffort-aware metrics (i.e., AUC, F -measure, and MCC) and one effort-aware metric (i.e., Popt) to evaluate the methods. Researchers in the SDP field have widely used these metrics. In addition, we employ the Wilcoxon signed-rank test and Cliff's δ to statistically analyze the prediction results of all methods.

8 | Conclusion

We propose UFR-OSFA, a novel MP-HDP approach that leverages unified feature representation and oppositional structure feature alignment. First, we match common features and employ the KS test and Hungarian algorithm to align unique features, minimizing distribution gaps. Next, we construct models comprising a generator and two classifiers with oppositional structures. Both the source project scattering classifier and the target project clustering classifier are built using the fully labeled source data together with a small set of labeled target data. The former maximizes conditional entropy to disperse the source features, whereas the latter minimizes conditional entropy to cluster the target features. This approach effectively reduces distribution mismatch and prevents decision boundary bias toward the source project.

Moreover, we perform an experimental evaluation of UFR-OSFA on 23 widely used projects from five common datasets. Compared with other baseline methods, UFR-OSFA achieves improvements of at least 8.7%, 7.5%, 16%, and 13% in AUC, F -measure, MCC and Popt, respectively. Correspondingly, we assess the statistical significance of the results using the Wilcoxon signed-rank test and measure the magnitude of differences between methods through Cliff's δ . The experimental findings demonstrate the superiority of UFR-OSFA in MP-HDP, and the statistical analysis confirms that our approach exhibits significant differences compared to other methods.

In future work, we will further validate the effectiveness of the UFR-OSFA method on a wider range of open-source and commercial projects. Additionally, we will employ automatic parameter optimization techniques to fine-tune the parameter settings and enhance model interpretability.

Acknowledgments

The research was supported by the National Natural Science Foundation of China (No. 62272126), the Fundamental Research Funds for the Central Universities, Special Project for Industrial Foundation

Reconstruction and High-Quality Development of Manufacturing Industry by the Ministry of Industry and Information Technology (No. TC220A04X-1), and the Natural Science Foundation of Heilongjiang (LH2021F001).

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. Z. Li, J. Niu, and X. Y. Jing, "Software Defect Prediction: Future Directions and Challenges," *Automated Software Engineering* 31, no. 1 (2024): 19.
2. Y. Zhao, K. Damevski, and H. Chen, "A Systematic Survey of Just-in-Time Software Defect Prediction," *ACM Computing Surveys* 55, no. 10 (2023): 1–35.
3. N. Grattan, D. A. Costa, and N. Stanger, "The Need for More Informative Defect Prediction: A Systematic Literature Review," *Information and Software Technology* 171 (2024): 107456.
4. Z. Kotti, R. Galanopoulou, and D. Spinellis, "Machine Learning for Software Engineering: A Tertiary Study," *ACM Computing Surveys* 55, no. 12 (2023): 1–39, <https://doi.org/10.1145/3572905>.
5. Y. Z. Bala, P. A. Samat, K. Y. Sharif, and N. Manshor, "Current Software Defect Prediction: A Systematic Review," in *2022 Applied Informatics International Conference (AiIC)* (2022), 117–121.
6. N. Li, M. Shepperd, and Y. Guo, "A Systematic Review of Unsupervised Learning Techniques for Software Defect Prediction," *Information and Software Technology* 122 (2020): 106287, <https://doi.org/10.1016/j.infsof.2020.106287>.
7. G. Giray, K. E. Bennin, Ö. Köksal, Ö. Babur, and B. Tekinerdogan, "On the Use of Deep Learning in Software Defect Prediction," *Journal of Systems and Software* 195 (2023): 111537, <https://doi.org/10.1016/j.jss.2022.111537>.
8. K. Zhu, S. Ying, N. Zhang, and D. Zhu, "Software Defect Prediction Based on Enhanced Metaheuristic Feature Selection Optimization and a Hybrid Deep Neural Network," *Journal of Systems and Software* 180 (2021): 111026.
9. Z. Xu, J. Liu, X. Luo, et al., "Software Defect Prediction Based on Kernel PCA and Weighted Extreme Learning Machine," *Information and Software Technology* 106 (2019): 182–200.
10. A. De and C. Guo, "An Adaptive Vector Quantization Approach for Image Segmentation Based on SOM Network," *Neurocomputing* 149 (2015): 48–58.
11. Y. Sun, X. Y. Jing, F. Wu, X. Dong, Y. Sun, and R. Wang, "Semi-Supervised Heterogeneous Defect Prediction With Open-Source Projects on GitHub," *International Journal of Software Engineering and Knowledge Engineering* 31, no. 06 (2021): 889–916, <https://doi.org/10.1142/s0218194021500273>.
12. S. Hosseini, B. Turhan, and D. Gunarathna, "A Systematic Literature Review and Meta-Analysis on Cross Project Defect Prediction," *IEEE Transactions on Software Engineering* 45, no. 2 (2019): 111–147, <https://doi.org/10.1109/tse.2017.2770124>.
13. H. Chen, X. Y. Jing, Z. Li, D. Wu, Y. Peng, and Z. Huang, "An Empirical Study on Heterogeneous Defect Prediction Approaches," *IEEE Transactions on Software Engineering* 47, no. 12 (2020): 2803–2822.
14. H. Tong, B. Liu, and S. Wang, "Kernel Spectral Embedding Transfer Ensemble for Heterogeneous Defect Prediction," *IEEE Transactions on Software Engineering* 47, no. 9 (2019): 1886–1906.
15. H. Chen, X. Y. Jing, Y. Zhou, B. Li, and B. Xu, "Aligned Metric Representation Based Balanced Multiset Ensemble Learning for Heterogeneous Defect Prediction," *Information and Software Technology* 147 (2022): 106892.
16. J. Niu, Z. Li, H. Chen, X. Dong, and X. Y. Jing, "Data Sampling and Kernel Manifold Discriminant Alignment for Mixed-Project Heterogeneous Defect Prediction," *Software Quality Journal* 30, no. 4 (2022): 917–951.
17. Y. Ren, B. Liu, and S. Wang, "Joint Instance and Feature Adaptation for Heterogeneous Defect Prediction," *IEEE Transactions on Reliability* 73, no. 1 (2023): 741–756.
18. P. R. Bal and S. Kumar, "A Data Transfer and Relevant Metrics Matching Based Approach for Heterogeneous Defect Prediction," *IEEE Transactions on Software Engineering* 49, no. 3 (2023): 1232–1245.
19. Z. Li, X. Y. Jing, and X. Zhu, "Heterogeneous Fault Prediction With Cost-Sensitive Domain Adaptation," *Software Testing, Verification and Reliability* 28, no. 2 (2018): e1658.
20. Z. Li, X. Y. Jing, X. Zhu, H. Zhang, B. Xu, and S. Ying, "On the Multiple Sources and Privacy Preservation Issues for Heterogeneous Defect Prediction," *IEEE Transactions on Software Engineering* 45, no. 4 (2019): 391–411, <https://doi.org/10.1109/TSE.2017.2780222>.
21. Y. Chen and H. Chen, "Landmark-Based Domain Adaptation and Selective Pseudo-Labeling for Heterogeneous Defect Prediction," *Electronics* 13, no. 2 (2024): 456.
22. L. Gong, S. Jiang, and L. Jiang, "Conditional Domain Adversarial Adaptation for Heterogeneous Defect Prediction," *IEEE Access* 8 (2020): 150738–150749.
23. X. Jing, F. Wu, X. Dong, F. Qi, and B. Xu, "Heterogeneous Cross-Company Defect Prediction by Unified Metric Representation and CCA-Based Transfer Learning," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (2015), 496–507.
24. K. Javed, R. Shengbing, M. Asim, and M. A. Wani, "Cross-Project Defect Prediction Based on Domain Adaptation and LSTM Optimization," *Algorithms* 17, no. 5 (2024): 175.
25. J. Nam, W. Fu, S. Kim, T. Menzies, and L. Tan, "Heterogeneous Defect Prediction," *IEEE Transactions on Software Engineering* 44, no. 09 (2018): 874–896.
26. A. Wang, Y. Feng, M. Yang, H. Wu, Y. Iwahori, and H. Chen, "Cross-Project Software Defect Prediction Using Differential Perception Combined With Inheritance Federated Learning," *Electronics* 13, no. 24 (2024): 4893.
27. H. Tong, D. Zhang, J. Liu, et al., "MASTER: Multi-Source Transfer Weighted Ensemble Learning for Multiple Sources Cross-Project Defect Prediction," *IEEE Transactions on Software Engineering* 50 (2024): 1281–1305.
28. H. Tong, W. Lu, W. Xing, and S. Wang, "ARRAY: Adaptive Triple Feature-Weighted Transfer Naive Bayes for Cross-Project Defect Prediction," *Journal of Systems and Software* 202 (2023): 111721, <https://doi.org/10.1016/j.jss.2023.111721>.
29. Y. Zou, H. Wang, H. Lv, S. Zhao, and H. Tian, "A Cross-Project Defect Prediction Approach Based on Code Semantics and Cross-Version Structural Information," *International Journal of Software Engineering and Knowledge Engineering* 34 (2024): 1–37.
30. J. Bai, J. Jia, and L. F. Capretz, "A Three-Stage Transfer Learning Framework for Multi-Source Cross-Project Software Defect Prediction," *Information and Software Technology* 150 (2022): 106985.
31. Z. Li, X. Y. Jing, X. Zhu, H. Zhang, B. Xu, and S. Ying, "Heterogeneous Defect Prediction With Two-Stage Ensemble Learning," *Automated Software Engineering* 26 (2019): 599–651.
32. J. Chen, X. Wang, S. Cai, J. Xu, J. Chen, and H. Chen, "A Software Defect Prediction Method With Metric Compensation Based on Feature Selection and Transfer Learning," *Frontiers of Information Technology & Electronic Engineering* 23, no. 5 (2022): 715–731.

33. J. Nam, S. J. Pan, and S. Kim, "Transfer Defect Learning," in *2013 35th international conference on software engineering (ICSE)* (2013), 382–391.
34. O. P. Omondiagbe, S. A. Licorish, and S. G. MacDonell, "Improving Transfer Learning for Software Cross-Project Defect Prediction," *Applied Intelligence* 54, no. 7 (2024): 5593–5616.
35. C. Qin, L. Wang, Q. Ma, Y. Yin, H. Wang, and Y. Fu, "Contradictory Structure Learning for Semi-Supervised Domain Adaptation," in *Proceedings of the 2021 SIAM International Conference on Data Mining* (2021), 576–584.
36. H. Song, G. Wu, L. Ma, Y. Pan, Q. Huang, and S. Jiang, "Adversarial Domain Adaptation for Cross-Project Defect Prediction," *Empirical Software Engineering* 28, no. 5 (2023): 127.
37. C. Qin, L. Wang, Q. Ma, Y. Yin, H. Wang, and Y. Fu, "Semi-Supervised Domain Adaptive Structure Learning," *IEEE Transactions on Image Processing* 31 (2022): 7179–7190, <https://doi.org/10.1109/TIP.2022.3215889>.
38. Q. Yu, S. Jiang, and Y. Zhang, "A Feature Matching and Transfer Approach for Cross-Company Defect Prediction," *Journal of Systems and Software* 132 (2017): 366–378.
39. Z. Li, X. Y. Jing, F. Wu, X. Zhu, B. Xu, and S. Ying, "Cost-Sensitive Transfer Kernel Canonical Correlation Analysis for Heterogeneous Defect Prediction," *Automated Software Engineering* 25 (2018): 201–245.
40. E. Kim, J. Baik, and D. Ryu, "Heterogeneous Defect Prediction Through Correlation-Based Selection of Multiple Source Projects and Ensemble Learning," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)* (2021), 503–513.
41. X. Liu, Z. Li, J. Zou, and H. Tong, "An Empirical Study on Multi-Source Cross-Project Defect Prediction Models," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)* (2022), 318–327.
42. Y. Tang, Q. Dai, Y. Du, L. Chen, and X. Niu, "A Software Defect Prediction Method Based on Learnable Three-Line Hybrid Feature Fusion," *Expert Systems With Applications* 239 (2024): 122409.
43. S. Zhang, S. Jiang, and Y. Yan, "A Hierarchical Feature Ensemble Deep Learning Approach for Software Defect Prediction," *International Journal of Software Engineering and Knowledge Engineering* 33, no. 04 (2023): 543–573.
44. C. Zhang, X. Wang, J. Chen, S. Cai, and R. Nii Ayitey Sosu, "A Novel Defect Prediction Method Based on Semantic Feature Enhancement," *Journal of Software: Evolution and Process* 36 (2024): e2674.
45. S. Qiu, M. Huang, Y. Liang, C. Peng, and Y. Yuan, "Code Multiview Hypergraph Representation Learning for Software Defect Prediction," *IEEE Transactions on Reliability* 73 (2024): 1863–1876.
46. S. Feng, J. Keung, Y. Xiao, P. Zhang, X. Yu, and X. Cao, "Improving the Undersampling Technique by Optimizing the Termination Condition for Software Defect Prediction," *Expert Systems With Applications*. 235 (2024): 121084.
47. X. Yu, J. Rao, L. Liu, et al., "Improving Effort-Aware Defect Prediction by Directly Learning to Rank Software Modules," *Information and Software Technology* 165 (2024): 107250.
48. Y. Qu and H. Yin, "Evaluating Network Embedding Techniques' Performances in Software Bug Prediction," *Empirical Software Engineering* 26, no. 4 (2021): 60.
49. P. He, B. Li, X. Liu, J. Chen, and Y. Ma, "An Empirical Study on Software Defect Prediction With a Simplified Metric Set," *Information and Software Technology* 59 (2015): 170–190.
50. R. Moussa and F. Sarro, "On the Use of Evaluation Measures for Defect Prediction Studies," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (2022), 101–113.
51. A. Marjuni, T. B. Adji, and R. Ferdiana, "Unsupervised Software Defect Prediction Using Signed Laplacian-Based Spectral Classifier," *Soft Computing* 23, no. 24 (2019): 13679–13690.
52. L. Gong, G. K. Rajbahadur, A. E. Hassan, and S. Jiang, "Revisiting the Impact of Dependency Network Metrics on Software Defect Prediction," *IEEE Transactions on Software Engineering* 48, no. 12 (2021): 5030–5049.
53. H. Wang, W. Zhuang, and X. Zhang, "Software Defect Prediction Based on Gated Hierarchical LSTMs," *IEEE Transactions on Reliability* 70, no. 2 (2021): 711–727.
54. Y. Xing, X. Qian, Y. Guan, B. Yang, and Y. Zhang, "Cross-Project Defect Prediction Based on G-LSTM Model," *Pattern Recognition Letters* 160 (2022): 50–57.
55. H. Tong, B. Liu, and S. Wang, "Software Defect Prediction Using Stacked Denoising Autoencoders and Two-Stage Ensemble Learning," *Information and Software Technology* 96 (2018): 94–111.
56. X. Chen, Y. Mu, Y. Qu, et al., "Do Different Cross-Project Defect Prediction Methods Identify the Same Defective Modules?," *Journal of Software: Evolution and Process* 32, no. 5 (2020): e2234.
57. S. Feng, J. Keung, P. Zhang, Y. Xiao, and M. Zhang, "The Impact of the Distance Metric and Measure on SMOTE-Based Techniques in Software Defect Prediction," *Information and Software Technology* 142 (2022): 106742.
58. X. Yu, J. Keung, Y. Xiao, S. Feng, F. Li, and H. Dai, "Predicting the Precise Number of Software Defects: Are We There Yet?," *Information and Software Technology* 146 (2022): 106847.
59. X. Chen, D. Zhang, Y. Zhao, Z. Cui, and C. Ni, "Software Defect Number Prediction: Unsupervised vs Supervised Methods," *Information and Software Technology* 106 (2019): 161–181.